

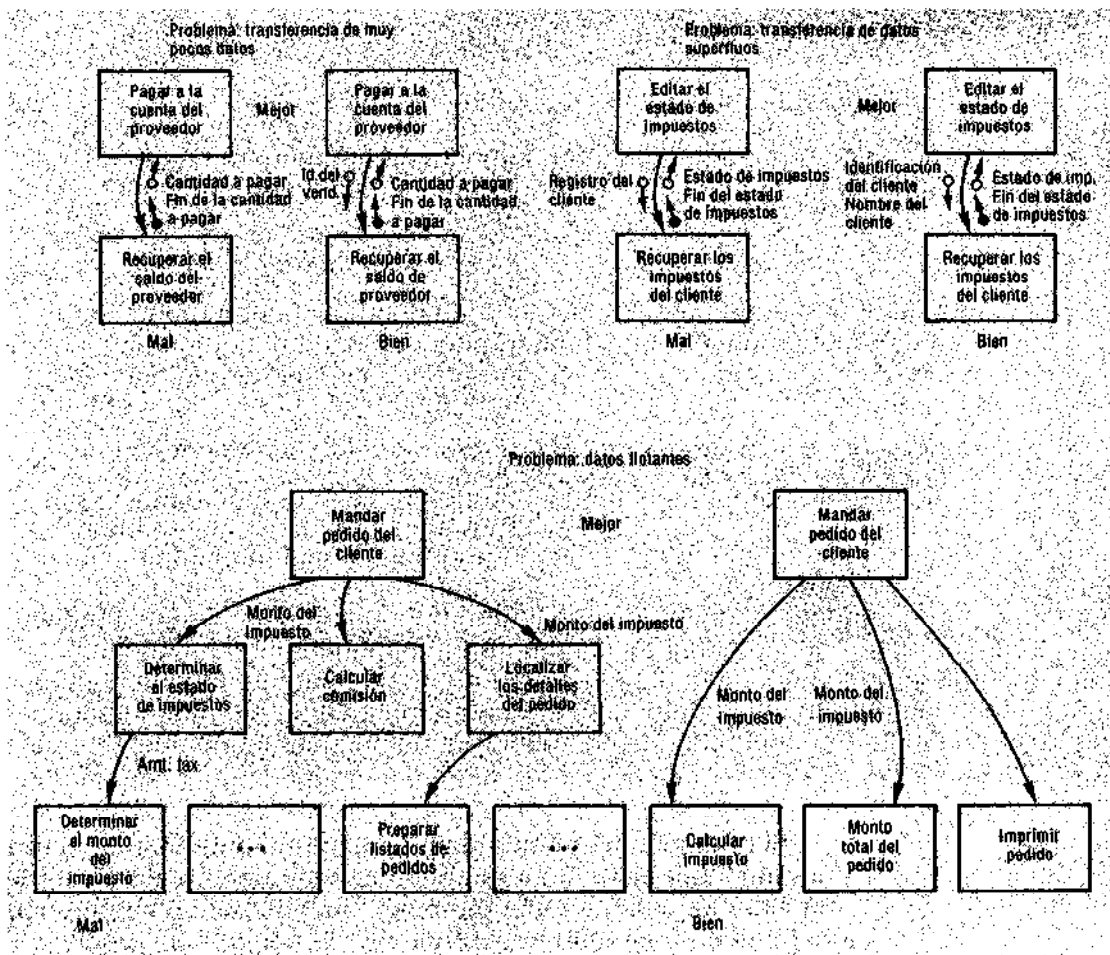


FIGURA 14.8
Ejemplos de diseño

función específica y para que sea más fácilmente entendible que los sistemas diseñados por otros métodos. Hay cuatro tipos generales de contenidos de módulos (Tabla 14.4); éstos se presentan enseguida en orden del menos al más deseable (Stevens, Myers y Constantine, 1974).

El tipo menos recomendable de agrupación del contenido de un módulo está formado por pasos que no llevan a cabo una función completa o que lógicamente no van juntos. Este caso extremo puede ocurrir si los programadores trabajan de acuerdo a reglas estrictas y dividen los módulos en secciones de 50 enunciados cada uno (o algún otro límite impuesto). Las actividades de entrada, salida, cálculo y manejo de archivos se llevan a cabo en un único módulo. Si esto ocurre, por lo general es el resultado del trabajo del programador sin especificaciones explícitas de diseño, o sin un claro entendimiento de cómo manejar una tarea.

	CONTENIDO DE UN MÓDULO	EXPLICACIÓN
MEJOR	Contenido de un módulo determinado por la función desarrollada	Todas las actividades en un módulo tienen el mismo propósito, es decir, llevar a cabo una función específica. <i>Ejemplos:</i> Calcular el cargo financiero en un estado de cuenta; editar una transacción.
	Contenido de un módulo determinado por los datos usados	Todos los elementos en un módulo se refieren a los mismos datos o archivos. <i>Ejemplos:</i> Imprimir, desplegar, y copiar datos de un archivo común.
	Contenido de un módulo determinado por la lógica del procesamiento	Todos los pasos se realizan juntos o manejan las mismas funciones. <i>Ejemplos:</i> Todas las operaciones de entrada, todas las operaciones de salida y todas las actividades de inicialización.
PEOR	Contenido no relacionado de un módulo	Los módulos se desarrollan por tamaño o número de instrucciones (es resultado de la programación con reglas escritas). <i>Ejemplos:</i> Todos los módulos no deben tener más de 50 enunciados de longitud; todos los módulos deben ajustarse a una sola página.

El contenido del módulo también se puede agrupar debido a que va junto lógicamente. Un módulo que maneja todas las operaciones de entrada o las de salida o uno que maneja actividades de procesamiento de pedidos, independientemente del tipo de cliente o las necesidades de manejo de datos por cada cliente, usa el agrupamiento lógico.

Los elementos del módulo también se pueden relacionar por el tiempo en el que ejecutan; es decir, lógicamente parecen ir juntos y se llevan a cabo al mismo tiempo. Un módulo que inicializa todas las variables y abre archivos está lógicamente integrado. Este nivel de modularidad es mejor que el primer tipo, ya que todos los elementos son ejecutables en un mismo marco de tiempo.

Los módulos que están lógicamente integrados son difíciles de modificar puesto que el costo será compartido por cada tipo de actividad. Hasta el más sencillo de los cambios puede afectar todos los tipos de transacción. Una mejor solución consiste en separar cada tipo de transacción en su propio módulo.

El contenido del módulo también se puede determinar por los datos que usen. Un módulo en el que el contenido se refiera a los mismos datos es preferible a uno que sea desarrollado sólo bajo la base de la lógica del procedimiento. Por ejemplo, se puede diseñar un módulo para que todas las operaciones sobre un conjunto de datos se

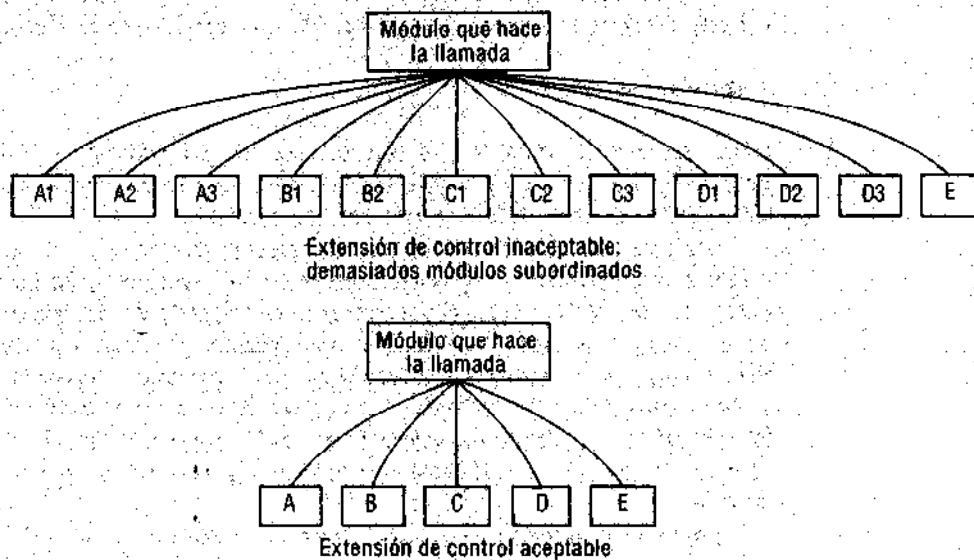


FIGURA 14.9
Extensión de control
en la jerarquía de un
módulo

Lleven a cabo al mismo tiempo: se prepara un archivo para imprimirse en papel, llevado a disco y también duplicado para fines de respaldo. Un módulo que lee la siguiente transacción y actualiza el archivo maestro, añadiendo, eliminando o cambiando registros, incluyendo la verificación de error necesaria para cada tipo de función, comparte un conjunto común de datos. Este tipo de cohesión es mejor que los otros tipos presentados, pero no tan bueno como el agrupamiento funcional.

El agrupamiento o cohesión funcional permite una prueba más global del módulo. Si se necesitan cambios posteriores, los analistas y programadores pueden determinar rápidamente cómo se estructura el módulo y cómo procesa los datos e interactúa con los demás módulos en el sistema.

Se enfatiza constantemente en la confiabilidad y facilidad de mantenimiento en todo el desarrollo del sistema.

Extensión de control

La *extensión de control* se refiere al número de módulos subordinados controlados por el módulo que hace la llamada. En general, hay que tratar de no tener más de cinco a siete módulos subordinados (Fig. 14.9).

Por otro lado, la excesiva extensión de control, lo que significa un alto número de módulos subordinados, crea un peso considerable cuando se desea determinar cuál módulo llamar bajo ciertas condiciones y al establecer secuencias de llamadas para transferir datos y

recibir resultados. Por otra parte, esto resulta común al no sujetarse a los objetivos de acoplamiento y cohesión presentados anteriormente.

Tamaño del módulo

¿Qué tan grande debe ser el módulo de un programa? Aunque es imposible fijar un número específico de instrucciones, existen algunos criterios útiles para definir el tamaño del módulo.

Algunas organizaciones han establecido reglas para el manejo del tamaño del módulo. Una regla común es que ningún módulo debe tener más de 50 instrucciones. Otra es que el listado del código fuente para un módulo debe limitarse a una página impresa. En algunas situaciones, estas reglas son adecuadas, pero en otras producen decisiones arbitrarias (por ejemplo, “si el módulo no puede codificarse en 50 instrucciones, crear un segundo módulo que sea llamado por el primero”) que restan coherencia a la decisión relativa al tamaño del módulo.

En general, debemos procurar diseños en donde los módulos tengan una función específica, sean altamente cohesivos y estén acoplados holgadamente. Sin embargo, los módulos no deben ser demasiado pequeños (cuando esto ocurre, deben conjuntarse los módulos). El tamaño del módulo depende también del lenguaje que se use. Más o menos, 50 enunciados en COBOL pueden ser un límite superior aceptable (los programadores no tienen que rediseñar si, digamos, se necesitan 51 o 55 enunciados, siempre que se cumplan los demás objetivos del diseño). Por otra parte, 50 instrucciones en un lenguaje poderoso de cuarta generación (donde una instrucción reemplaza de 10 a 50 enunciados equivalentes en COBOL) serían probablemente inaceptables.

Módulos compartidos

El uso compartido surge del deseo de tener una cierta función, cálculo o tipo de proceso llevado a cabo en un lugar del sistema. Se desea diseñar el sistema de forma que un cierto cálculo (como determinar los impuestos por ventas en un sistema de procesamiento de pedidos) se lleve a cabo una sola vez. Así, el módulo puede compartirse por todo el sistema llamándolo desde varios módulos.

¿Por qué compartir módulos? Hay varias razones. Primera, el compartir módulos minimiza la cantidad de software que debe diseñarse y escribirse. Segunda, minimiza el número de cambios que hay que hacer durante el mantenimiento del sistema. Por ejemplo, si cambian los procedimientos de cálculo de impuestos, solamente hay que modificar un módulo si se utiliza el principio de módulos compartidos. Y tercera, al tener un único módulo compartido se reduce la probabilidad de error: hay una probabilidad muy alta de que los módulos redundantes sigan distintos procedimientos de cálculo, si

bien no inicialmente, sino después de la introducción de cambios en el mantenimiento (un módulo puede no ser cambiado porque se haya perdido de vista).

Muchos sistemas establecen los módulos de biblioteca —procedimientos predefinidos— que se incluyen en la biblioteca de programas del sistema. La rutina se invoca rápidamente mediante un solo comando o llamada.

Los módulos de biblioteca compartidos son otra forma en la que los buenos diseñadores siguen los principios introducidos al principio de esta sección.

DISEÑO DE SOFTWARE Y HERRAMIENTAS DE DOCUMENTACIÓN

El software modular y bien diseñado tiene mayor probabilidad de cumplir con los requerimientos de facilidad de mantenimiento, confiabilidad y prueba delineados en el presente capítulo. En esta sección se estudian tres herramientas específicas: los diagramas de flujo estructurado, diagramas de HIPO y diagramas de Warnier/Orr.

Diagramas de flujo estructurado

Los *diagramas de flujo estructurado*, también llamados de *Nassi-Schneiderman*, son herramientas gráficas que fuerzan al diseñador a estructurar software que sea modular y descendente. Proporcionan una estructura a la que se pueden ajustar los que desarrollan el software de aplicación. Las responsabilidades en una organización varían. En algunas compañías, los analistas son responsables de desarrollar la lógica del módulo, mientras que en otras esa responsabilidad se delega en el programador. En cualquier caso, el programador debe estar preparado en el uso de los diagramas de flujo estructurado.

Elementos básicos

Existen tres elementos básicos usados en el desarrollo de los diagramas de flujo estructurado: proceso, decisión e iteración. (Hay muchas analogías entre estos elementos y los componentes que se usan en el español estructurado.)

Proceso Los procesos o pasos en un programa se representan mediante un rectángulo: el símbolo de proceso. Este símbolo representa la inicialización de variables, actividades de entrada y salida y las llamadas para ejecutar otros procedimientos.

Un nombre descriptivo breve, escrito dentro del rectángulo, establece el propósito del proceso. La serie de pasos se muestra usando varios rectángulos.

Decisión El símbolo de decisión representa condiciones alternativas que pueden ocurrir y que el programa debe poder manejar. Muestran el equivalente de las estructuras IF-THEN-ELSE, estudiadas con anterioridad, en el caso del español estructurado y que son comunes a varios lenguajes de programación. Como se verá en los ejemplos, el símbolo de decisión puede mostrar acciones para más de dos alternativas al mismo tiempo.

Iteración El símbolo de iteración representa los ciclos y repetición de operaciones *mientras* exista una condición dada o *hasta* que haya una condición. La forma del símbolo de iteración muestra claramente el alcance de la iteración, incluyendo todos los procesos y decisiones contenidos dentro del ciclo. La parte izquierda del símbolo representa la ruta de repetición a seguir hasta que las condiciones que controlan la iteración se satisfagan.

Uso de diagramas de flujo estructurado

Los diagramas de flujo estructurado no usan flechas o continuaciones en páginas separadas. Cada diagrama de flujo estructurado se muestra en una sola hoja de papel (o una sola pantalla, si se desarrolla en línea).

Al diseñar un diagrama de flujo estructurado, los analistas de sistemas especifican la lógica en una forma descendente. El primer elemento a considerar en un proceso o decisión es el elemento de arriba. El segundo en la serie es el siguiente que se muestre, y así sucesivamente. En forma análoga, existe una única salida del proceso.

El analista comienza con un proceso principal e introduce otros símbolos para subdividir el proceso. Cada proceso recibe un nombre, pero, si el nombre no se subraya, es una referencia a otro diagrama o descripción. Esta sencilla convención hace posible conjuntar fácilmente distintos procedimientos que se llevan a cabo para terminar una actividad compleja.

La figura 14.10 muestra los pasos necesarios para producir las facturas de los clientes durante el procesamiento mensual. El diagrama de flujo estructurado se lee de arriba hacia abajo y de izquierda a derecha. Cada actividad se anida dentro de la iteración y procesos alternativos del cual es parte. Además, se muestra claramente cada condición.

Algunas partes de los procesos a menudo se describen en diagramas de menor nivel. Por ejemplo, las transacciones de compra, pago, débito y crédito se señalan en el módulo que se describe en la figura 14.10, pero los módulos que manejan el procesamiento de cada tipo de transacción se aplican aparte.

Un uso de los diagramas de flujo estructurado importante para el diseñador que debe verificar las especificaciones del sistema contra la lógica planeada del software, es el de identificar las condiciones y los procedimientos a seguir cuando existan dichas condiciones. El hecho

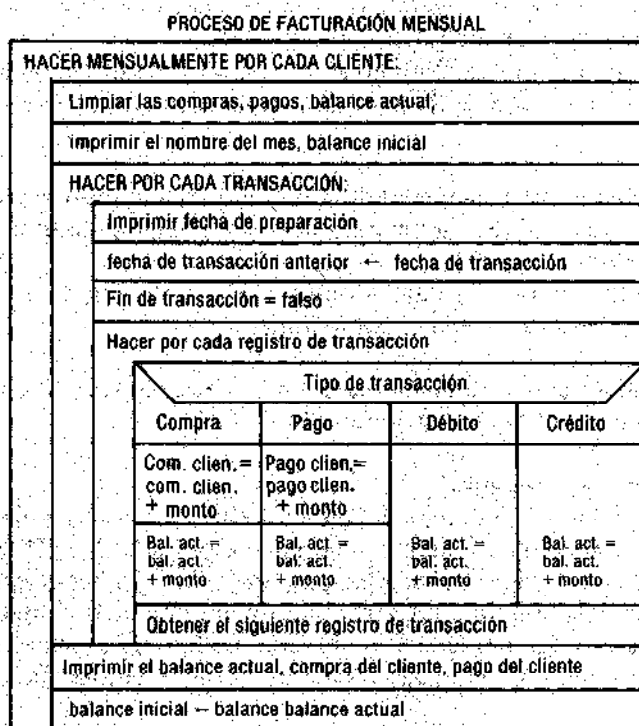


FIGURA 14.10
Diagrama de estructura para la facturación mensual

de que el diagrama de flujo estructurado sea fácil de leer permitirá al analista determinar si la transacción de ajuste de débito, por ejemplo, fue añadida por el programador o era parte de las especificaciones originales del sistema.

HIPO

HIPO es otro método de uso común para desarrollar software de sistemas. *HIPO* es una abreviatura del nombre en inglés de la *entrada-proceso-salida-jerarquica*, método que fue desarrollado por IBM para sus sistemas operativos grandes y complejos.

Propósito

La hipótesis en la que *HIPO* se basa es que es fácil perder la pista de la función deseada de un sistema o componente de un sistema grande. Esta es una razón por la que es difícil comparar los sistemas existentes contra sus especificaciones originales (y por lo tanto, porque pueden ocurrir fallas incluso en los sistemas técnicamente bien formulados). Desde el punto de vista del usuario, una sola función puede a menudo extenderse a varios módulos, por lo tanto, el interés del analista es entender, describir y documentar los módulos y su interacción de

forma que se obtenga el detalle suficiente, pero que no se pierda de vista el panorama general.

Los diagramas de HIPO son descripciones gráficas del sistema, en vez de prosa o narrativa. Ayudan al analista a responder tres preguntas guía:

1. *¿Qué hace el sistema o módulo?* (Se pregunta al diseñar el sistema.)
2. *¿Cómo lo hace?* (Se interroga al revisar el código para su prueba o mantenimiento.)
3. *¿Cuáles son las entradas y salidas?* (Se pregunta al revisar el código para su prueba o mantenimiento.)

Una descripción de HIPO para un sistema consta de una tabla visual de contenidos y los diagramas funcionales.

Tabla visual de contenidos

La *tabla visual de contenidos (VTOC)* muestra la relación entre cada uno de los documentos que conforma un paquete de HIPO. Está formada por un diagrama de jerarquía que identifica los módulos en un sistema mediante un número y en relación con los otros (Fig. 14.11) y da una descripción breve de cada módulo. Los números en la sección de contenido se relacionan con los de la sección de organización.

Los módulos aparecen en detalle creciente. Según la complejidad del sistema, son típicos de 3 a 5 niveles de módulos.

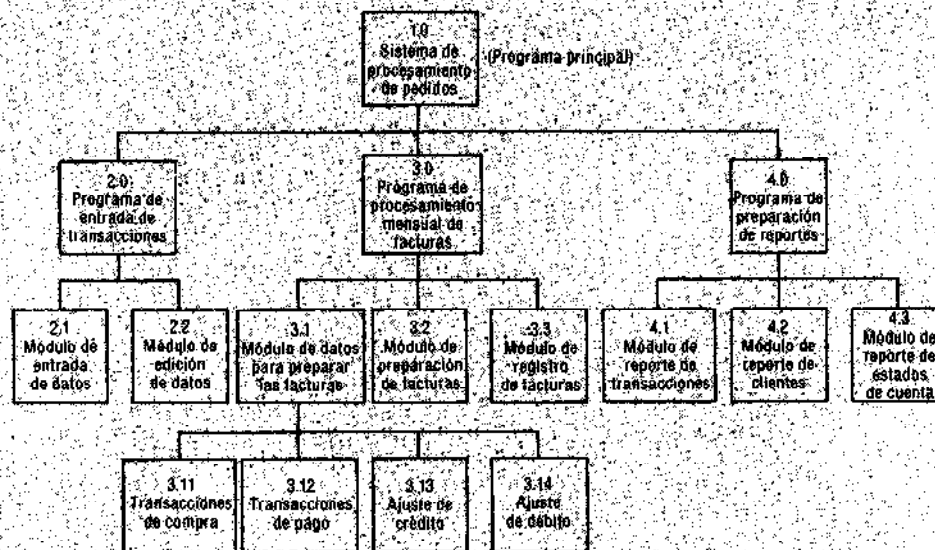
Diagramas funcionales

Existe un diagrama por cada caja en el VTOC. Cada diagrama muestra la entrada y salida (de derecha a izquierda o de arriba hacia abajo), los procesos principales, movimiento de datos y puntos de control. Los símbolos de los diagramas de flujo tradicionales representan los medios, tales como cinta magnética, disco magnético y salida impresa. Una flecha sólida muestra las rutas de control y una flecha hueca identifica el flujo de datos.

La ilustración de la figura 14.11 es un panorama del sistema, que representa la caja del nivel más alto en el VTOC. Los números de la parte inferior del diagrama indican donde se halla más detalle; es decir, se refieren a otros números de documento. Por ejemplo, el documento 3.11 explica el proceso de transacción de compra con mayor detalle.

Algunos diagramas funcionales contienen otros diagramas intermedios (Fig. 14.12). Pero también muestran datos externos, al igual que datos desarrollados internamente (como las tablas en el ejemplo de las facturas) y el paso en el procedimiento donde se usan los datos. Se le puede añadir una descripción de un diccionario de datos para explicar aún más los elementos de datos usados en un proceso.

Los diagramas de HIPO son efectivos para documentar un sis-



CONTENIDO

- | | |
|--|--|
| <p>1.0 SISTEMA DE PROCESAMIENTO DE PEDIDOS (PROGRAMA PRINCIPAL)
Controla todo el procesamiento. Llama programas para manejar la entrada de datos, el procesamiento mensual de los registros y la impresión de reportes.</p> <p>2.0 PROGRAMA DE ENTRADA DE TRANSACCIONES
Controla la entrada de todos los datos y la edición de los datos ya almacenados. Incluye las funciones de compra, pago y ajuste.</p> <p>2.1 MÓDULO DE ENTRADA DE DATOS
Lleva a cabo la entrada y validación de datos para comprar, pagos y ajustes de crédito o débito.</p> <p>2.2 MÓDULO DE EDICIÓN DE DATOS
Lleva a cabo la recuperación y edición de los datos de transacción previamente analizadas para comprar, pagos y ajustes de crédito o débito. Permite cambios a los datos o a la eliminación del registro de transacción.</p> <p>3.0 PROGRAMA DE PROCESAMIENTO MENSUAL DE FACTURAS
Controla todos los pasos del procesamiento de facturas usando los datos de transacciones introducidos durante el mes.</p> <p>3.1 MÓDULO DE DATOS PARA PREPARAR LAS FACTURAS
Lleva a cabo el procesamiento mensual de los datos de las transacciones para preparar la impresión de las facturas. Llama a módulos separados para manejar las transacciones de compra, pago y ajuste.</p> | <p>3.2 MÓDULO DE PREPARACIÓN DE FACTURAS
Imprime las facturas en formas ya impresas. Debe hacerse sólo hasta después de que los datos de la factura han sido preparados por el módulo 3.1.</p> <p>3.3 MÓDULO DE REGISTRO DE FACTURAS
Imprime una relación detallada de facturas numeradas en forma ascendente. Debe hacerse sólo hasta después de que los datos de la factura hayan sido preparados por el módulo 3.1.</p> <p>4.0 PROGRAMA DE PREPARACIÓN DE REPORTES
Controla toda la impresión de un reporte para los reportes de transacción, cliente y estado de cuenta.</p> <p>4.1 MÓDULO DE REPORTE DE TRANSACCIONES
Imprime un listado detallado de todas las transacciones introducidas durante el mes, ordenado según el número de transacción asignado.</p> <p>4.2 MÓDULO DE REPORTE DE CLIENTES
Imprime un listado de todos los clientes y la información demográfica en orden alfabético y de número de cuenta.</p> <p>4.3 MÓDULO DE REPORTE DE ESTADOS DE CUENTA
Imprime los listados de todas las cuentas con balance distinto de cero en orden descendente. Incluye la información cronológica.</p> |
|--|--|

FIGURA 14.11

Tabla visual de contenido HIPO.

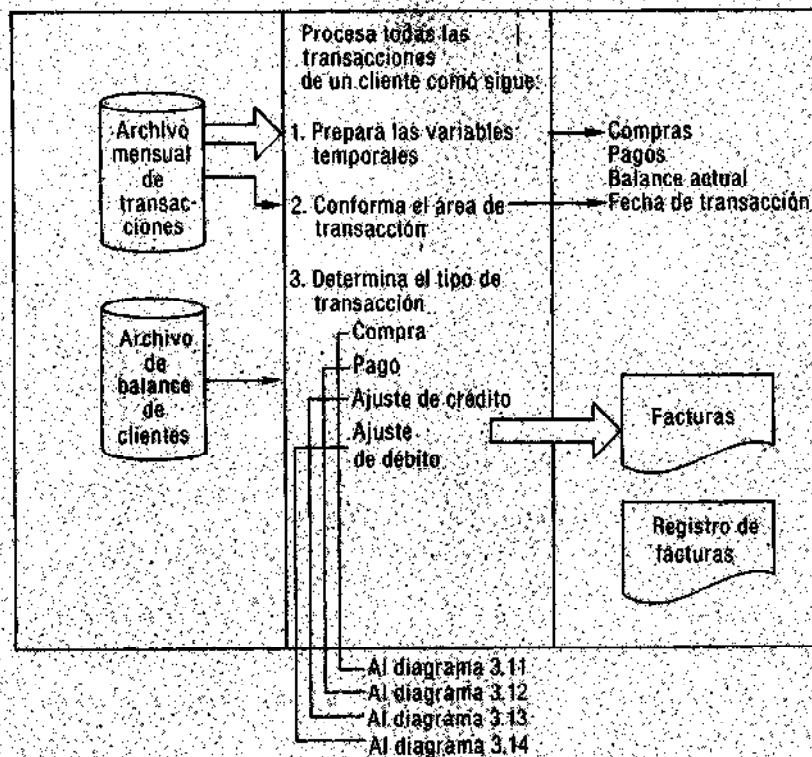


FIGURA 14.12

Diagrama funcional de HIPO para el procesamiento mensual de facturas.

tema. También ayuda a los diseñadores y los fuerzan a pensar en cómo cumplir con las especificaciones y dónde hay que ligar las actividades y componentes. Sin embargo, se basan en un conjunto de símbolos especializados que requieren explicación, una preocupación adicional si se compara con la simplicidad de, por ejemplo, los diagramas de flujo de datos. Los diagramas de HIPO no son tan fáciles de usar para los fines de comunicación como mucha gente quisiera. Y, por supuesto, no garantizan sistemas libres de errores. Por lo tanto, su mayor fuerza es la documentación de un sistema.

Diagramas de Warnier/Orr

Los diagramas de Warnier/Orr (también conocidos como *construcción lógica de programas/construcción lógica de sistemas*) fueron desarrollados inicialmente en Francia por Jean-Dominique Warnier y en los Estados Unidos por Kenneth Orr. Este método ayuda al diseño de estructuras de programas identificando la salida y resultado del procedimiento, y entonces trabaja hacia atrás para determinar los pasos y combinaciones de entrada necesarios para producirlos. Los sencillos métodos gráficos usados en los diagramas de Warnier/Orr hacen evi-

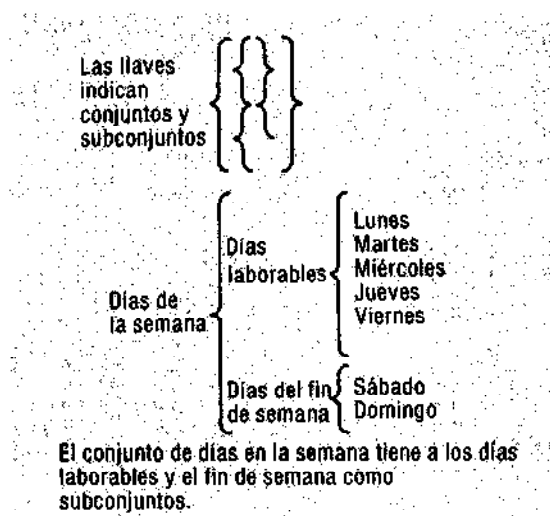


FIGURA 14.13
Simbología de conjuntos usada en los diagramas de Warnier/Orr.

dentes los niveles en un sistema y más claros los movimientos de los datos en dichos niveles.

Elementos básicos

Los diagramas de Warnier/Orr muestran los procesos y la secuencia en que se realizan. Cada proceso se define de una manera jerárquica; es decir, consta de conjuntos de subprocesos que lo definen. En cada nivel, el proceso se muestra en una llave que agrupa a sus componentes (Fig. 14.13). Puesto que un proceso puede tener muchos subprocesos distintos, un diagrama de Warnier/Orr usa un conjunto de llaves para mostrar cada nivel del sistema.

Los factores críticos en la definición y desarrollo del software son la iteración o repetición y la selección de alternativas. Los diagramas de Warnier/Orr muestran esto muy bien. Por ejemplo, al desarrollar un sistema de facturación, existen procesos que ocurren en cada transacción, cada día y cada mes. Como lo muestra la figura 14.14, la repetición de cada caso no sólo es fácil de identificar sino también se localiza rápidamente en una secuencia lógica.

En algunas situaciones, la única preocupación es si está presente cierta característica. Las alternativas son que esté o no esté. La simbología que se usa para indicar que una condición no existe es una línea sencilla sobre el nombre de la condición. Por ejemplo, en la figura 14.15 los cuatro tipos de transacción de compra, pago, ajuste del débito y ajuste del crédito son alternativas. El símbolo + representa alternativas.

Uso de diagramas de Warnier/Orr

La capacidad de mostrar la relación entre procesos y pasos de un proceso no es exclusiva de los diagramas de Warnier/Orr, así como

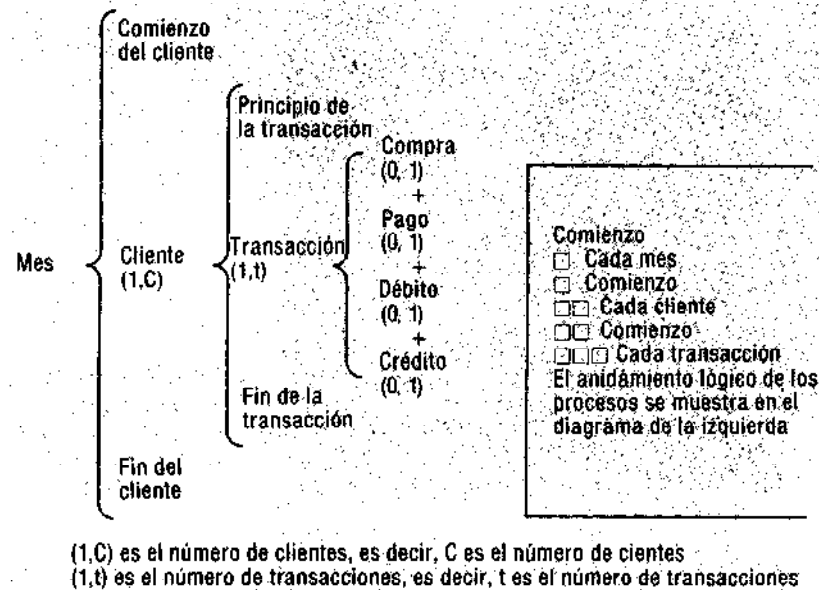


FIGURA 14.14
Seguimientos en el procesamiento mensual de facturas

tampoco lo es el uso de la iteración, selección de alternativas o el tratamiento de casos individuales. Tanto los diagramas de flujo estructurado y los métodos del español estructurado logran esto también. Sin embargo, el enfoque que se usa para desarrollar las definiciones de un sistema por medio de diagramas de Warnier/Orr es distinto y se adapta bien a los que se usan en el diseño de sistemas lógicos.

Para desarrollar un diagrama de Warnier/Orr, el analista trabaja hacia atrás, empezando con la salida del sistema y usando un análisis orientado hacia la salida. En el papel el desarrollo se mueve de izquierda a derecha. En primer lugar, se definen la salida o resultados esperados del procedimiento. En el nivel siguiente, mostrado mediante la inclusión por medio de una llave, se definen los pasos necesarios para producir la salida. A su vez, cada paso se define un poco más. Las llaves adicionales agrupan los procesos requeridos para producir el resultado en el siguiente nivel.

Un diagrama completo de Warnier/Orr incluye las agrupaciones de procesos y los requerimientos de datos. Como lo indica la figura 14.15, los elementos de los datos se enlistan para cada proceso o componente del proceso. Estos elementos de datos son los necesarios para determinar cuál alternativa o caso se debe manejar por el sistema y llevar adelante el proceso. El analista tiene que determinar dónde se origina cada elemento dato, cómo se usa y cómo se combinan los elementos individuales. Al terminar la definición, se documenta una estructura de datos para cada proceso. Ésta, a su vez, es usada por los

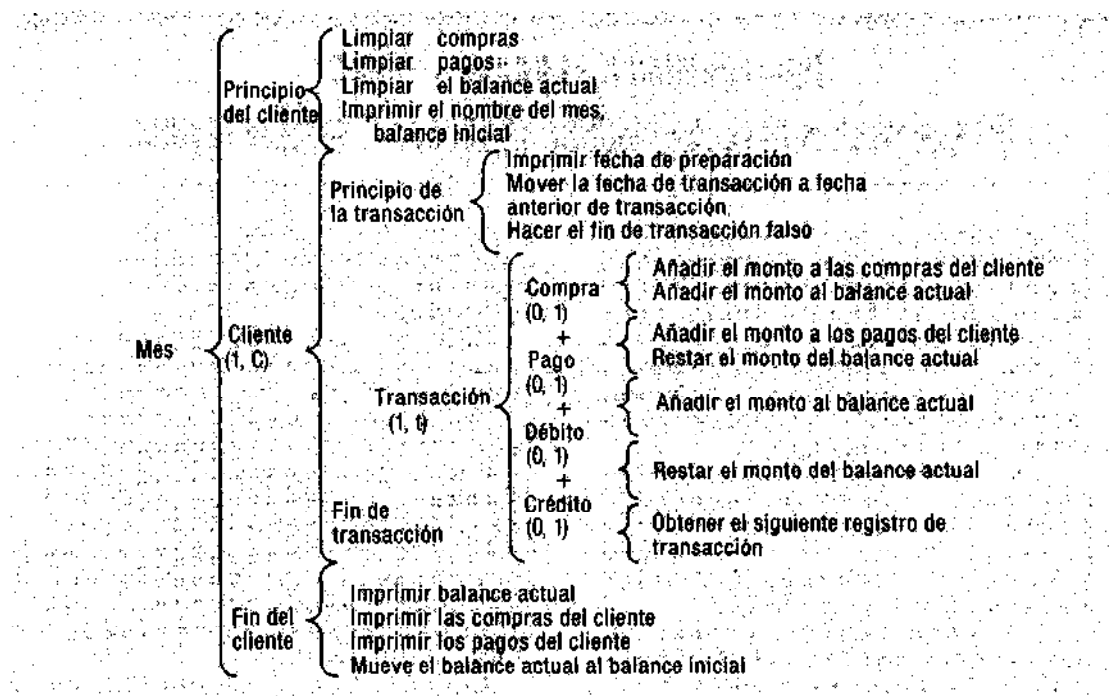


FIGURA 14.15

Datos que se usan en el procesamiento mensual de las facturas

programadores, los que trabajan a partir de los diagramas para codificar el software.

Los diagramas de Warnier/Orr ofrecen a los expertos en sistemas algunas ventajas distintivas. Son simples en apariencia y fáciles de entender. Aún así, son poderosas herramientas de diseño. Tienen la ventaja de mostrar agrupaciones de procesos y los datos que deben transferirse de nivel a nivel. Además, la secuencia del trabajo hacia atrás garantiza que el sistema estará orientado hacia el resultado. Este método también es un proceso natural. Con los diagramas de flujo estructurado, por ejemplo, a menudo es necesario determinar los pasos más internos antes de poder resolver lo relativo a las iteraciones y la modularidad.

Este método es útil para definir los datos y los procesos. Se puede usar para cada uno independientemente o se pueden combinar ambos en el mismo diagrama, como se hizo en el ejemplo anterior.

Una característica poderosa de los diagramas de Warnier/Orr es la forma en que se muestran los ciclos. Los pasos necesarios para moverse de un mes al otro en el procesamiento de facturas, por ejemplo, se señalan en la figura. Los pasos del procesamiento y los pasos de preparación de reportes se muestran con claridad, de tal forma que quien use el diagrama puede visualizar rápidamente cuándo se ejecutan los pasos y cuáles artículos están listos para terminar un ciclo.

INGENIERÍA HACIA ATRÁS: REENCARNACIÓN DE UN SISTEMA

La mayoría de los aproximadamente 750 000 analistas de sistemas y programadores de los Estados Unidos trabajan en sistemas de información existentes. El mantenimiento de los sistemas de información es una enorme tarea en prácticamente cualquier organización. A nivel nacional, se invierten aproximadamente 55 billones de dólares al año para dar mantenimiento a los sistemas existentes.

Los analistas están prestando cada vez más atención al desarrollo de métodos, herramientas y técnicas para dar mantenimiento a los sistemas existentes. La ingeniería hacia atrás se está convirtiendo en un método valioso a considerar para llevar a cabo esto.

La ingeniería hacia atrás es el proceso de crear especificaciones, las cuales describan la acción de las aplicaciones existentes, en contraste con la ingeniería hacia adelante, en la que las especificaciones se crean de lo general a lo específico y que son seguidas por el desarrollo. La ingeniería hacia atrás también se puede llevar a cabo para rehacer la información que describe el propósito de los sistemas existentes.

La ingeniería hacia atrás es un concepto importante que se adecúa a la evolución de los sistemas de información. En la mayoría de las corporaciones, los sistemas no mueren; se les redirige, lo cual significa que los componentes se vuelven a desarrollar y usar para crear nuevas aplicaciones.

La documentación que describe en forma completa a los sistemas de información, así como los formatos de archivos y bases de datos, y el código fuente de los programas, es un cuerpo importante de detalles que explica cómo funciona un sistema, y en algunos casos, por qué se le ha diseñado de esa manera. Los comentarios en el código del programa y la

información que se incluye en reportes con cierto formato también proporcionan pistas importantes para hallar la lógica oculta en algunos programas.

Todo esto pudiera parecer misterioso, casi como introducirse por la puerta trasera de una casa solitaria. Y hay cierta verdad en eso. En muchas compañías, la persona o equipo que desarrolló el sistema puede haberse cambiado a otra empresa por lo que no está disponible para explicar por qué un sistema se desarrolló de cierta manera incluso cómo funciona. La ingeniería hacia atrás proporciona dicha información crítica para su uso en la modificación o rediseño del sistema o para encontrar formas para poder usar los componentes en otros sistemas.

No existen hasta el momento métodos estándares para llevar a cabo la ingeniería hacia atrás, aunque éstos llegarán algún día. Se espera que evolucionen las herramientas automatizadas para la ingeniería hacia atrás, permitiendo que los usuarios interactúen con un sistema para transferir el software en especificaciones. Es probable que un sistema de pregunta-respuesta permita consultas sobre, digamos, dónde se usa un cierto grupo de datos, dónde entra al sistema y en qué forma se genera. También es muy probable que ciertas herramientas produzcan bosquejos gráficos de la interrelación entre componentes, como un mapa de carreteras que une puntos clave.

La capacidad para ayudar a los analistas a desarrollar los sistemas con ingeniería hacia atrás podría convertirse en un importante elemento para mejorar la productividad del desarrollo de las aplicaciones nuevas y existentes.

mensual y prepararse para el siguiente. Este aspecto del procesamiento del sistema es crítico en un sistema de calidad.

MANEJO DEL PROCESO PARA GARANTIZAR LA CALIDAD

Las técnicas y criterios presentados en la sección anterior producirán, en caso de aplicarse adecuadamente, sistemas bien diseñados y con pocas fallas. Sin embargo, aun cuando las técnicas se sigan cuidadosamente, el analista no debe suponer que se ha cumplido con los estándares de calidad necesarios, punto que se enfatizó en el ejemplo del principio del capítulo.

El aseguramiento de la calidad es la revisión de los productos y documentación relacionada con el software para verificar su cobertura, corrección, confiabilidad y facilidad de mantenimiento. Y, por supuesto, incluye la garantía de que un sistema cumple las especificaciones y los requerimientos para su uso y desempeño deseados.

Niveles de seguridad

Los analistas usan cuatro niveles de aseguramiento de la calidad: prueba, verificación, validación y certificación.

Prueba

La prueba del sistema es un proceso caro pero crítico que puede llevarse hasta 50% del presupuesto para el desarrollo del programa. El punto de vista común respecto a las pruebas compartido por los usuarios, es que se lleva a cabo para demostrar que no hay errores en un programa. Sin embargo, como se indicó anteriormente, esto es prácticamente imposible, puesto que los analistas no pueden demostrar que el software está limpio de errores.

Por lo tanto, el enfoque más útil y práctico es en el entendimiento de que la prueba es el proceso de ejecutar un programa con la intención explícita de hallar errores, es decir, hacer que el programa falle. El examinador, que puede ser un analista, programador, o especialista entrenado en la prueba de software, está tratando realmente de hacer que el programa falle. Así, una prueba exitosa es aquella que encuentra un error.

Los analistas saben que un programa de prueba efectivo no garantiza la confiabilidad del sistema. La confiabilidad es asunto del diseño, como se estudió en este capítulo. Por lo tanto, la confiabilidad debe diseñarse en el sistema. Los analistas no pueden hacer una prueba de ella. Posteriormente, en esta misma sección, se discutirán estrategias específicas de prueba.



Comentario al margen

Estimación de la confiabilidad de los sistemas: el cliente tiene el control

En las empresas, un dicho común entre los coordinadores y los representantes de ventas es “el cliente siempre tiene la razón”. Últimamente, la frase, aunque no el significado, ha cambiado: “el cliente tiene el control”, lo cual quiere decir que el cliente determina lo que es correcto y aceptable.

Esto es similar en los sistemas de información. Los usuarios son los clientes y los usuarios tienen el control. Determinan si un sistema es bueno o no. Cualquier intento de un analista por convencer a los usuarios de lo sabio o correcto de un cierto enfoque no sirve de nada si los usuarios no piensan que el sistema funciona de manera razonable.

Cuando hay que decidir si un sistema es confiable, la decisión es muy simple. Si los usuarios —aquellos que usarán el sistema semana tras semana— no están convencidos que hace lo correcto (y en la forma correcta, sin sorpresas), no es un sistema confiable. Como dicen, ¡el cliente tiene el control!

Verificación y validación

Como la prueba, la *verificación* tiene la intención de hallar errores. Se lleva a cabo ejecutando un programa en un ambiente simulado. La *validación* se refiere al proceso del uso del software en un ambiente no simulado para hallar sus errores.

Cuando los sistemas comerciales se desarrollan con la intención explícita de distribuirlos a través de terceros para su venta, o comercializarlos por medio de oficinas de la propia compañía, primero deben pasar por la verificación, a veces llamada *prueba alfa*. La retroalimentación de la fase de validación generalmente produce cambios en el software para resolver los errores y fallas que se descubren. Se elige un conjunto de instalaciones usuarias que ponen a trabajar el sistema en un ambiente real. Estas instalaciones de *prueba beta* usan el sistema en las actividades cotidianas; procesan transacciones en directo y producen salidas normales del sistema. El sistema está a prueba en toda la extensión de la palabra, excepto que los usuarios están advertidos de que están usando un sistema que puede fallar. Sin embargo, las transacciones que se están procesando y las personas que usan el sistema son reales.

Es posible continuar la validación durante varios meses. En el curso de la validación del sistema, puede ocurrir una falla y el software será modificado. El uso continuo posiblemente produzca fallas adicionales y la necesidad de más cambios.

COMPANÍA DE AUDITORES CERTIFICADOS

123 Wall Street
New York, New York 10015

CERTIFICACIÓN DE SOFTWARE

Hemos revisado el paquete de servicio médico de la Medical Office Associates Company y elaborado un informe que describe nuestra revisión. Las observaciones principales de nuestro informe son que, en nuestra opinión, el paquete tiene la documentación adecuada tanto en el nivel del sistema como en el de la aplicación, lo cual permite a sus usuarios potenciales que entiendan el paquete. Proporciona controles internos de procesamiento, informes y archivos de datos que son adecuados para permitir el desarrollo de auditorías, de acuerdo con los principios y estándares generalmente aceptados en los Estados Unidos.

El sistema procesa la información de contabilidad utilizando procedimientos que permiten a los usuarios producir informes que se ajustan a los principios de contabilidad que generalmente son aceptados en los Estados Unidos.

Estos reportes y sus anexos son para el paquete de servicio médico proporcionado por Medical Office Associates Company.

FIGURA 14.16

Informe de
certificación de
software

Certificación

La *certificación* del software es una garantía de lo correcto de un programa, su importancia va en aumento para las aplicaciones de sistemas de información. Existe una creciente dependencia de la compra o renta de software comercial en vez del desarrollo "en casa". Sin embargo, antes que los analistas deseen aprobar la adquisición de un paquete, a menudo requieren de la certificación del software por parte del fabricante o de un tercero sin prejuicios.

Por ejemplo, algunas empresas importantes de contabilidad están involucradas en la certificación de paquetes de software, para garantizar que realmente hace lo que el vendedor afirma que realiza y de una manera apropiada. Para certificar de esta forma el software, la empresa asigna a un equipo de especialistas que cuidadosamente examinan la documentación del sistema para determinar lo que afirma el vendedor que el sistema hace y cómo lo lleva a cabo. Entonces ellos prueban el software contra estas afirmaciones. Si no se encuentran serias discrepancias o fallas, certificarán que el software hace lo que la documentación afirma (Fig. 14.16). Sin embargo, no certifican que el software sea el paquete correcto para una cierta organización. Esta responsabilidad sigue siendo de la organización y su equipo de analistas.

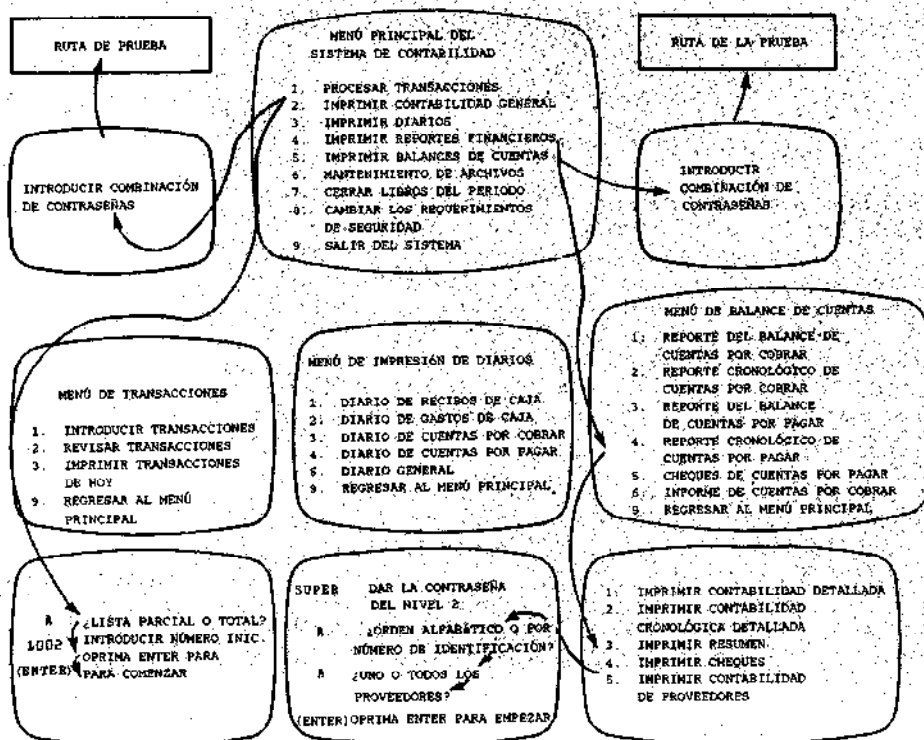


FIGURA 14.17

Datos para probar múltiples rutas en un sistema de contabilidad.

Estrategias de prueba

Ya se ha indicado que la filosofía detrás de la prueba es la de encontrar errores. Los casos de prueba se diseñan con este propósito en mente. Un *caso de prueba* es un conjunto de datos que el sistema procesará como entrada normal. Sin embargo, los datos se crean con la intención expresa de determinar si el sistema los procesará correctamente. Por ejemplo, los casos de prueba para el manejo de inventarios deben incluir situaciones en las que las cantidades que se retiran del inventario excedan, igualen y sean menores que las existencias. Cada caso de prueba se diseña con la intención de encontrar errores en la forma en que el sistema los procesará.

Hay dos estrategias generales para la prueba del software. Esta sección examina ambas, las estrategias de prueba de código y prueba de especificación.

Prueba de código

La estrategia de *prueba de código* examina la lógica del programa. Para seguir este método de prueba, el analista desarrolla casos de prueba que produzcan la ejecución de cada instrucción en el programa

o módulo; es decir, se prueba cada ruta del programa. Una ruta es una combinación específica de condiciones manejadas por el programa. Por ejemplo, en el caso del sistema de contabilidad, una ruta por el sistema es la de cambiar los balances de las cuentas. Se introduce la solicitud correcta, en seguida las contraseñas, datos y comandos adecuados (Fig. 14.17).

A primera vista, la prueba de código parece ser un método ideal para probar el software. Sin embargo, como vimos en el ejemplo del principio de este capítulo, es incorrecto el razonamiento de que todos los errores de software se pueden descubrir verificando toda ruta en un programa. En primer lugar, incluso en los programas moderadamente grandes, del tamaño usado en las situaciones típicas de un negocio, es casi imposible hacer una prueba exhaustiva de esta naturaleza. Las consideraciones financieras y las limitaciones de tiempo por lo general impedirán la ejecución de cada ruta dentro de un programa, ya que puede haber varios miles de ellas.

Sin embargo, aunque la prueba de código se pueda llevar a cabo en su totalidad, no es una garantía en contra de las fallas del software. Esta estrategia de prueba no indica si el código cumple sus especificaciones ni tampoco determina si todos los aspectos han sido implantados. La prueba de código tampoco verifica el rango de los datos que aceptará el programa, aun cuando, al ocurrir fallas del software en su uso real, sea frecuente que los usuarios introduzcan datos fuera de los rangos esperados (por ejemplo, una orden de ventas por un millón de dólares —la más grande en la historia de la organización—).

Prueba de especificación

Para llevar a cabo la *prueba de especificación* el analista examina las especificaciones que señalan lo que el programa debe hacer y cómo lo debe llevar a cabo bajo diferentes condiciones. Después se desarrollan casos de prueba para cada condición o combinación de condiciones y se mandan para su procesamiento. Por medio del estudio de los resultados, el analista puede determinar si el programa funciona de acuerdo con los requerimientos especificados.

Esta estrategia trata al programa como si fuera una caja negra: el analista no mira dentro del programa para estudiar el código y no le interesa si se prueba cada instrucción o ruta dentro del programa. En ese sentido, la prueba de especificación no es una prueba completa. Sin embargo, la hipótesis es que si el programa cumple las especificaciones, no fallará.

Ninguna de las estrategias de prueba de código o especificación es ideal. Sin embargo, la prueba de especificación es la estrategia más eficiente, ya que se centra en la forma que se espera se use el software. También muestra de nuevo qué tan importantes son las especificaciones desarrolladas por los analistas durante todo el proceso de desarrollo del sistema.

MANEJO DE LAS PRÁCTICAS DE PRUEBAS

Independientemente de cual estrategia siga el analista, hay ciertas prácticas preferidas para garantizar que la prueba sea útil. Los niveles de prueba y los tipos de datos de prueba, junto con las bibliotecas de prueba, son aspectos importantes del proceso real de prueba.

Niveles de prueba

Los sistemas no se diseñan como sistemas completos ni tampoco se prueban como sistemas únicos. El analista debe llevar a cabo tanto pruebas parciales como las del sistema.

Pruebas parciales

En las pruebas parciales, el analista prueba los programas que conforman a un sistema. (Por esta razón, a veces se llama a las pruebas parciales *prueba de programas*, en contraste con la prueba de sistemas, que se estudia en la siguiente sección.) Las unidades de software en un sistema son los módulos y rutinas que se ensamblan e integran para llevar a cabo una función específica. En un sistema grande, se necesitan muchos módulos en varios niveles.

Las pruebas parciales se centran primero en los módulos, independientes entre sí, para localizar los errores. Esto permite al que realice la prueba detectar errores en el código y lógica contenidos dentro de ese único módulo. Aquellos errores que resultan de la interacción entre los módulos se evitan inicialmente.

Por ejemplo, un sistema de información de un hotel está formado por módulos para manejar las reservaciones; entrada y salida de huéspedes, restaurant, servicio al cuarto y cargos varios, actividades de convención y elaboración de estados de cuenta. Para cada uno, se tiene la capacidad de introducir, cambiar o recuperar datos y responder a consultas o imprimir reportes.

Los casos de prueba necesarios para las pruebas parciales deben probar cada condición u opción. Por ejemplo, los casos de prueba se necesitan para determinar cómo maneja el sistema los intentos, para registrar a clientes que tengan o no reservación, lo mismo que aquellos casos en los que hay que cambiar el nombre en la reservación cuando llega una persona distinta a la esperada. También se necesitan casos de prueba para las situaciones en las que el cliente paga el monto exacto de la factura, sólo parte de ella o más del monto mostrado. Es más, debe incluirse en un caso de prueba la salida del cliente sin que realice pago alguno.

Si el módulo recibe una entrada o genera una salida, también se necesitan casos de prueba para examinar el rango de valores esperado, incluyendo los datos válidos e inválidos. ¿Qué ocurrirá en el ejemplo de la salida del cliente del hotel si un cliente desea hacer un pago de \$ 150 000 para una próxima convención? ¿Están diseñados los módu-

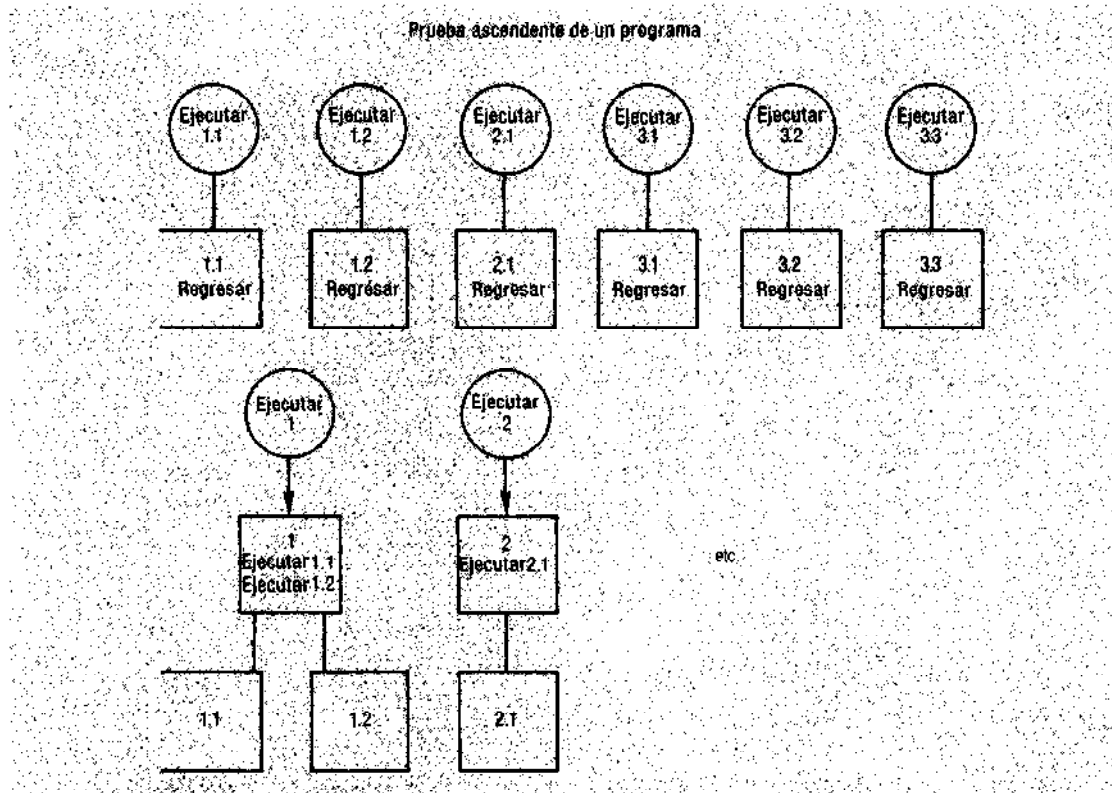


FIGURA 14.18
Método alternativo
para las pruebas
parciales usando un
enfoque ascendente

los de pago e impresión para manejar este monto? La prueba para esta pregunta detecta rápidamente los errores existentes.

Si el módulo se diseña para llevar a cabo iteraciones, con procesos específicos contenidos dentro de un ciclo, es recomendable ejecutar cada condición frontera: cero iteraciones, una iteración en el ciclo y el máximo número de iteraciones en el ciclo. Por supuesto, siempre es importante examinar los resultados de la prueba, pero hay que prestar especial atención a estas condiciones. Muy a menudo, los analistas cometen el error de suponer que el caso de cero iteraciones será manejado automáticamente en forma adecuada.

Las pruebas parciales se pueden llevar a cabo en forma ascendente, comenzando con los módulos más pequeños y de nivel inferior y continuando de uno en uno. Para cada módulo en la *prueba ascendente*, un programa corto (llamado *programa conductor* ya que maneja o corre el módulo) ejecuta el módulo y proporciona los datos necesarios, de esta forma se pide que el módulo se desempeñe en la forma en que lo haría al encajarse dentro del sistema. Después de probar los módulos de nivel inferior, la atención se centra en los del siguiente nivel (Fig. 14.18) que usan los de nivel inferior. Se prueban en forma

Prueba descendente de un programa.

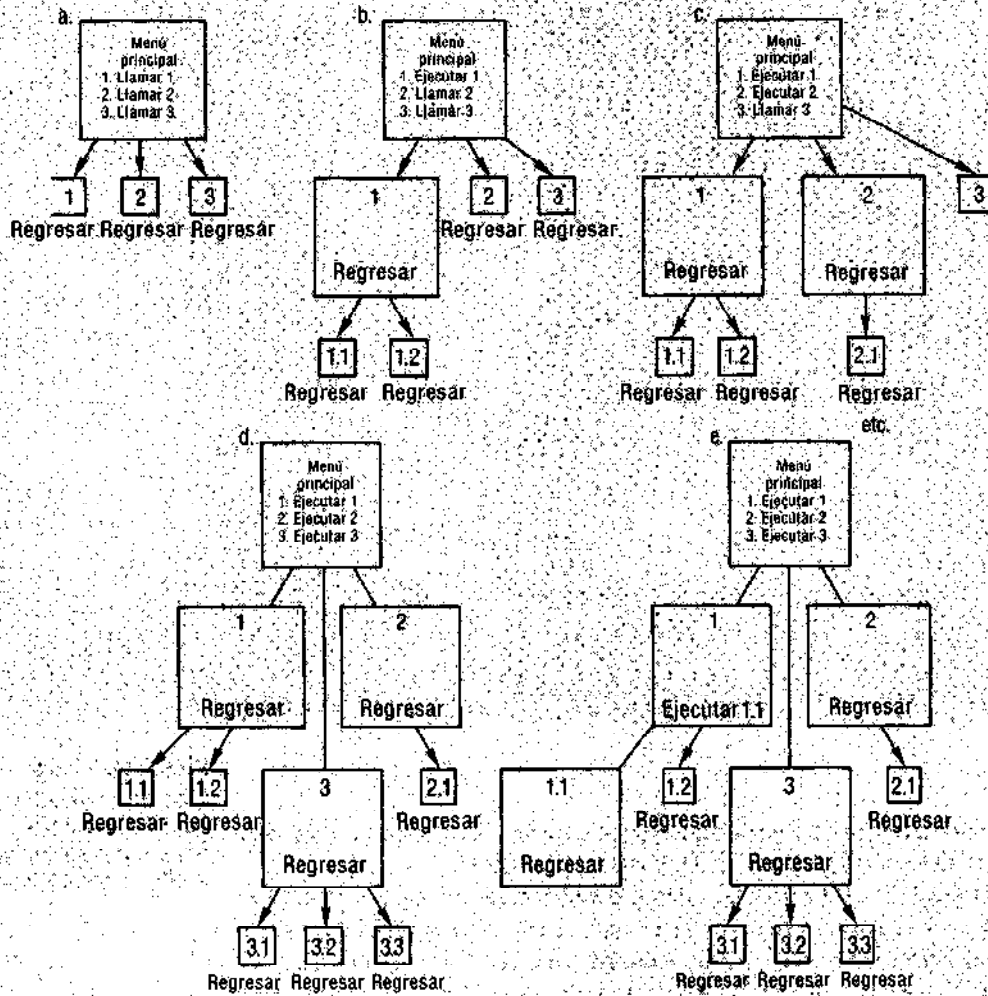


FIGURA 14.19

Método alternativo de prueba usando un enfoque descendente

individual y después conjuntamente con los módulos de nivel inferior que fueron examinados con anterioridad.

La *prueba descendente*, como su nombre lo indica, empieza con los módulos de nivel superior. Sin embargo, puesto que no se cuenta con las actividades de detalle, las que usualmente se llevan a cabo en las rutinas de nivel inferior (ya que esas rutinas no se están probando), se escriben fragmentos. Un fragmento es un módulo que puede ser llamado por el módulo de nivel superior y que, al ser alcanzado en forma apropiada, regresará un mensaje al módulo que hace la llamada, indicando que ocurrió la interacción apropiada (Fig. 14.19). No se hace el intento por verificar si el módulo del nivel inferior es correcto.

TABLA 14.3 Pruebas especiales de sistemas

TIPO DE PRUEBA	DESCRIPCIÓN
Prueba de carga máxima	Determinar si el sistema manejará el volumen de actividades que ocurran cuando el sistema esté en el punto más alto de su demanda de procesamiento. <i>Ejemplo:</i> Todas las terminales se activan al mismo tiempo.
Prueba de almacenamiento	Determinar la capacidad del sistema para almacenar datos de transacciones en un disco u otros archivos. <i>Ejemplo:</i> Verificar la documentación en el sentido de que el sistema almacenará 10 000 registros de 383 bytes de longitud en un único disco flexible.
Prueba de tiempo de ejecución	Determinar el tiempo de máquina que el sistema necesita para procesar los datos de una transacción. <i>Ejemplo:</i> Tiempo de respuesta a las consultas cuando el sistema está cargado en su totalidad con datos operativos.
Prueba de recuperación	Determinar la capacidad del usuario para recuperar los datos o restablecer el sistema después de una falla. <i>Ejemplo:</i> Cargar copias de respaldo de datos y reiniciar el procesamiento sin perder datos o integridad.
Prueba de procedimientos	Determinar la claridad de la documentación en los aspectos de operación y uso de un sistema, haciendo que los usuarios lleven a cabo exactamente lo que el manual pide. <i>Ejemplo:</i> Apagar el sistema al final de la semana o responder al indicador luminoso de la impresora que señala la falta de papel.
Prueba de factores humanos	Determinar como utilizarán los usuarios el sistema al procesar datos o preparar informes. <i>Ejemplo:</i> Reacciones del usuario cuando no hay una respuesta inmediata a una consulta.

A menudo, los planes de prueba descendente se conjuntan con las pruebas ascendentes; es decir, se prueban como unidad los módulos de nivel inferior y se integran a un programa de prueba descendente.

Prueba de sistemas

La prueba de sistemas no prueba el software en sí, sino la integración de cada módulo en el sistema. También busca las discrepancias entre el sistema y su objetivo original, especificaciones y documentación del sistema. La preocupación principal es la compatibilidad de los módulos individuales.

Los analistas tratan de hallar áreas en donde los módulos hayan sido diseñados con especificaciones distintas para la longitud y tipo de datos y los nombres de los elementos de los datos. Por ejemplo, un

módulo puede esperar que el tipo de dato para la identificación de un cliente sea un campo numérico, mientras que otros módulos esperan que sea de tipo carácter. Es posible que el sistema en sí no reporte esto como un error, pero la salida puede mostrar resultados inesperados. Si un registro creado y guardado en un módulo, usando la identificación como tipo numérico, se busca posteriormente esperando que sea de tipo carácter, el campo no será reconocido y aparecerá el mensaje EL REGISTRO PEDIDO NO SE ENCUENTRA.

La prueba de sistemas también debe verificar que los tamaños de los archivos son adecuados y que los índices se han construido en forma adecuada. Se deben probar a nivel sistema los procedimientos de ordenamiento y reindexación, que se supone están presentes en los módulos de nivel inferior, para ver que realmente existen y que logran los resultados que esperan los módulos.

Pruebas especiales de sistemas

Existen otras pruebas en una categoría especial ya que no se centran en el funcionamiento normal del sistema. Hay 6 pruebas básicas (Tabla 14.5).

Prueba de carga máxima Existen tiempos críticos en muchos sistemas, particularmente en los sistemas en línea. Por ejemplo, en un sistema bancario, los analistas quieren saber que ocurriría si todos los cajeros prenden sus terminales al mismo tiempo antes de empezar las actividades del día. ¿Los manejará el sistema uno a la vez sin incidentes?, ¿tratará de manejarlos todos a la vez y se confundirá tanto que se “cierre” y deba ser arrancado de nuevo?, ¿se perderán las direcciones de las terminales? La única forma segura para descubrirlo es probarlo. Las mismas situaciones pueden surgir cuando los cajeros apagan la terminal durante los descansos para comer y al final del día, por lo que la prueba se refiere a situaciones reales.

Prueba de almacenamiento Los analistas especifican una capacidad del sistema cuando se diseña y elabora. La capacidad se mide en términos del número de registros que un disco puede manejar o que un archivo puede contener. Estas capacidades están ligadas al espacio en disco y al tamaño de los índices, claves de registro y demás. Pero éstos también deben ser probados. Si la documentación de un sistema nuevo que va a correr en una microcomputadora afirma que un archivo en disco almacena más de 10 000 registros, cada uno con 393 bytes de longitud, la afirmación debe ser verificada antes de la implantación.

La prueba de almacenamiento requiere almacenar continuamente datos (ver las notas que siguen sobre los datos reales contra los sintéticos) hasta que se alcanza la capacidad. Al comparar las capacidades ofrecidas y reales se verificará, por un lado, la exactitud de la documentación y permitirá al mismo tiempo dar un juicio acerca de la

capacidad real. La mayoría de los sistemas no se prueban de esta forma. Los usuarios encuentran demasiado tarde que las afirmaciones hechas durante la instalación no son ciertas: no existe la capacidad suficiente de almacenamiento para las transacciones y los registros del archivo maestro.

Prueba del tiempo de ejecución Cuando los analistas están desarrollando un diseño, se preocupan más por los reportes, entradas, archivos y secuencias de proceso que por el tiempo de ejecución, aunque esto cambia con la experiencia. Durante las pruebas parciales y del sistema, se usan conjuntos relativamente pequeños de datos para hallar errores o provocar fallas. Por lo tanto, los usuarios con frecuencia saben qué tan rápido o lento es el sistema sólo hasta después de que ha sido instalado y cargado con los datos. Esto puede ocurrir demasiado tarde. (Rara vez, los sistemas son muy rápidos para los usuarios.)

La prueba del tiempo de ejecución se lleva a cabo antes de la implantación para determinar cuánto tiempo se lleva recibir una respuesta a una consulta, hacer una copia de respaldo de un archivo o mandar una transmisión y recibir una respuesta. También incluye corridas de prueba para conocer el tiempo necesario para indexar o reordenar grandes archivos del tamaño de los que tendrá el sistema durante una corrida típica, o bien preparar un reporte.

Un sistema que corre bien con sólo algunas transacciones de prueba puede ser inaceptablemente lento cuando esté cargado por completo. El momento para saber esto es antes de la implantación, cuando se pueden hacer con mayor facilidad los ajustes. Una vez que los archivos están cargados en su totalidad y el usuario confía en el sistema para sus actividades cotidianas, es difícil retirarlo y comenzar cambios a gran escala. El usuario necesita el sistema y el analista no quisiera arriesgar la pérdida de datos reales.

Prueba de recuperación Los analistas deben suponer siempre que el sistema fallará y que los datos se dañarán o perderán. Aunque se escriban planes y procedimientos para cubrir estas situaciones, también se deben probar. Mediante la creación de un evento de falla o pérdida de datos, en donde los usuarios se vean forzados a volver a cargar y recuperar a partir de una copia de respaldo, los analistas pueden determinar fácilmente si los procedimientos de recuperación son adecuados. Generalmente, aun los planes mejor diseñados se ajustan o aumentan después de esta prueba.

Prueba de procedimientos Los manuales de documentación y ejecución que dicen al usuario como llevar a cabo ciertas funciones se prueban fácilmente pidiendo al usuario que los siga en forma exacta por medio de una serie de eventos. Es sorprendente cómo pueden surgir preguntas ante el hecho de no incluir instrucciones sobre

cuándo oprimir la tecla ENTER, cuándo quitar los diskettes antes de apagar la máquina o qué hacer cuando se prende la luz que indica la falta de papel en la impresora.

Por supuesto, no hay sustituto para un conjunto de manuales de procedimiento bien diseñados. Los analistas se concentran en los detalles principales y críticos del diseño de un sistema y los incluyen en la documentación. También ponen atención en los pequeños detalles, tales como los mencionados anteriormente, al diseñar el sistema. Pero es común que las descripciones de los detalles no estén dentro de la documentación. Este tipo de prueba no sólo muestra dónde se necesitan, sino también en qué lugar están equivocados, es decir, dónde las acciones sugeridas en la documentación no son compatibles con las que realmente hay que llevar a cabo para hacer que el sistema funcione.



Comentario al margen

Documentación: para preservar la credibilidad

Lo peor que puede ocurrir cuando se implanta un nuevo sistema es que el software no funcione. Los profesionales de los sistemas de información se preocupan mucho por esta posibilidad, como se ha señalado en este capítulo.

Sin embargo, algo casi tan malo es que en la documentación los usuarios lean una cosa y que el sistema haga otra. En ese momento, tanto la aplicación como el analista pierden credibilidad. (Y una vez perdida la credibilidad es difícil de volverse a ganar.)

La documentación que va a dar a las manos del usuario final, ya sea en la forma de manuales, sistemas de ayuda en línea o diagramas sencillos, debe coincidir con la forma en que funcione la aplicación. Los analistas tienen que asegurarse de que ponen la atención suficiente para mantener este aspecto de credibilidad.

Prueba de los factores humanos ¿Qué hacen los usuarios si, después de mandar una transacción por medio de una terminal, la pantalla se pone en blanco mientras que los datos se procesan? Podrían no llevar a cabo las acciones que el analista desea o espera y en vez de ello responder de manera inusual: pueden oprimir la tecla de envío varias veces, apagar la terminal y volverla a prender, desconectar el teléfono y volverlo a conectar, volver a llamar al centro de cómputo o golpear la terminal. Obviamente, ellos harían cualquier cosa si el analista no les diera algún mensaje en la pantalla para indicar que su petición ha sido recibida, que está siendo procesada y que tardará un poco. De esto trata la prueba de factores humanos —hallar respuestas a preguntas sobre como reaccionará la gente ante el sistema en formas no previstas—. Como regla general, aunque parezcan extrañas las accio-

nes anteriores, la gente tiene la razón, llevan a cabo acciones que son normales bajo las circunstancias.

Es responsabilidad del analista el anticipar preguntas que surgirán en la mente de los usuarios cuando éstos interactúen con el sistema. Si una pantalla se pone en blanco durante el procesamiento de la transacción, el analista debe asegurarse de que aparezca un mensaje en el cual se informe al usuario que la computadora está trabajando. Incluso no es suficiente si el retraso es de más de un segundo o dos. Para el procesamiento que se tarde periodos largos, el analista debe hacer que la pantalla dé al usuario un mensaje en el que se dice aproximadamente *cuánto tiempo* se tardará y dando *una opción para cancelar la petición*. El usuario puede decidir correr ese trabajo de una hora en otro momento, cuando el sistema no esté tan ocupado.

Si el sistema va a hacer un proceso largo de ordenamiento, el analista debe mantener al usuario informado acerca de que proporción del ordenamiento ha terminado. Los usuarios aprecian los sistemas que muestran el número de registros ordenados o el porcentaje terminado.

También el analista debe asegurarse de *observar* cómo introducen los datos las personas. ¿Usan teclas diferentes de las anticipadas (tales como la parte superior de los números en el teclado de la máquina de escribir en vez del teclado numérico)? ¿Hay combinaciones difíciles de teclas que puedan provocar errores (por ejemplo, tener que mantener oprimida la tecla de mayúsculas con el dedo meñique mientras se presiona la tecla + con el índice)?

¿Cómo se sentirá el usuario de un sistema después de trabajar con él durante mucho tiempo? El resplandor de una pantalla o simplemente bastante detalle en la misma provoca irritación física y mental. Las ligeras modificaciones en el contenido de la pantalla o la distribución del equipo son preocupaciones importantes del factor humano que afectan de forma dramática al usuario y, por lo tanto, al sistema.

Estas preguntas simples de prueba son de considerable importancia y extremadamente útiles para hallar defectos que puedan provocar la falla del sistema. Algunos analistas hallarán estos defectos en la forma más difícil —por medio de malas experiencias—. No es fácil olvidar al sistema que fue dañado porque el usuario golpeó la terminal cuando introdujo los datos y fueron aceptados por el sistema sin mostrar una respuesta. Sin embargo, si sigue los criterios anteriores, el analista puede evitar dichas situaciones.

Diseño de datos de prueba

Hay dos fuentes muy diferentes de datos de prueba, reales y artificiales. Ambos tienen ventajas y desventajas para el que realiza la prueba.

Uso de datos de prueba reales

Los datos de prueba reales son aquellos que se extraen de los archivos

de la organización. Después de que un sistema está terminado parcialmente, es frecuente que los programadores o analistas pidan a los usuarios que introduzcan un conjunto de datos de sus actividades normales. Por ejemplo, en un sistema de contabilidad, le pueden pedir a alguien del personal de contabilidad que introduzca la relación de cuentas y los estados financieros, junto con transacciones que afecten a estas cuentas. El analista de sistemas usa estos datos para probar parcialmente el sistema. En otros casos, los programadores o analistas obtienen datos reales de los archivos y los introducen ellos mismos.

Es difícil obtener datos reales en cantidad suficiente para conducir una prueba extensa. Y, aunque los datos reales mostraran cómo funciona el sistema para los requerimientos típicos de procesamiento, suponiendo que los datos reales introducidos son en realidad típicos, generalmente dichos datos no probarán todas las combinaciones o formatos que puedan entrar al sistema. El sesgo hacia los valores típicos no proporciona entonces una verdadera prueba del sistema y de hecho ignora los casos más probables que causan la falla del mismo.

Uso de datos de prueba artificiales

Los datos de prueba artificiales se crean solamente con fines de prueba, ya que se pueden generar para probar todas las combinaciones de formatos y valores. En otras palabras, los datos artificiales, que se puedan preparar rápidamente mediante un programa de utilería para generación de datos en el departamento de sistemas de información, hacen posible la prueba de todas las rutas lógicas y de control en todo el programa.

Los programas de prueba más efectivos usan datos de prueba artificiales generados por personas distintas de las que escribieron los programas. Frecuentemente, un equipo independiente formula un plan de prueba, usando las especificaciones del sistema.

Bibliotecas de prueba

Para garantizar que todos los sistemas se prueban adecuadamente, muchas organizaciones establecen bibliotecas de prueba. Una *biblioteca de prueba* es un conjunto de datos desarrollados para probar en su totalidad un sistema. Se guarda en una forma fácil de leer por la máquina, usualmente en disco magnético, y se usa por todas las personas involucradas con un sistema particular.

Por ejemplo, un sistema grande de inventarios está formado por cientos de programas. Todos comparten datos y formatos de archivo comunes. Cada uno de ellos también procesará transacciones similares y a veces actualizará registros y en otras ocasiones recuperará datos para responder a consultas o prepara reportes y documentos. Puesto que estos programas son transacciones interdependientes y sus

procesos están relacionados, tiene sentido usar un conjunto común de datos para probar cada programa.

Las bibliotecas de prueba no solamente son para las pruebas iniciales. Al evolucionar el sistema y modificarse y dar mantenimiento a los programas, deben volverse a probar. La biblioteca de prueba tiene que conservarse durante toda la vida de un sistema de forma que, al hacer cada cambio, se disponga nuevamente de datos confiables para probar el sistema.

RESUMEN

La calidad de un sistema de información depende de su diseño, desarrollo, prueba e implantación. Un aspecto de la calidad del sistema es su *confiabilidad*. Un sistema es confiable si, al usarse de manera razonable, no produce fallas peligrosas o costosas. Esta definición distingue entre los *errores* del software, en los que el sistema no produce los resultados esperados, y las *fallas* que se presentan. Aunque es prácticamente imposible desarrollar software que se pueda demostrar que está libre de errores, los analistas hacen lo posible por prevenir la aparición de errores, usando métodos y técnicas que incluyen la detección de errores y su corrección y la tolerancia a errores. Ambas estrategias son útiles para mantener al sistema en operación y prevenir las fallas. A diferencia del hardware, en el que puede haber fallas de fabricación y del equipo, las fallas del software son el resultado de errores de diseño introducidos cuando se formularon las especificaciones y se escribió el software.

Un aspecto adicional del *aseguramiento de la calidad* es el evitar la necesidad de mejoras, por un lado, y por otra parte desarrollar software que sea fácil de mantener. La necesidad de mantenimiento es muy alta e impide los desarrollos nuevos. El porcentaje más alto del mantenimiento es para las mejoras propuestas por los usuarios y para mejorar la documentación —tareas que se pueden evitar o al menos reducir mediante una ingeniería de sistemas apropiada—.

Los *diagramas de estructura* son una herramienta importante para el diseñador de software. Muestran visualmente la relación entre los módulos del programa y los datos comunicados entre cada módulo. En general, se transmiten dos tipos de datos. Los *parámetros* son aquellos datos que el módulo necesita para el procesamiento. Por el contrario, la *información de control* ayuda a controlar el proceso, indicando la ocurrencia de errores o condiciones que afectan el proceso, tal como el indicador de fin de archivo.

Seis principios caracterizan a los buenos diseños de software: fragmentación descendente, *acoplamiento* holgado, agrupación funcional para la *cohesión*, extensión de control limitado, tamaño de adecuado de los módulos y módulos compartidos. Al seguir estos

principios se aumenta la probabilidad de alcanzar niveles aceptables de confiabilidad y facilidad de mantenimiento.

Las necesidades de mantenimiento y de aseguramiento de la calidad también se cumplen mejor cuando se usa cualesquiera de las tres herramientas de desarrollo estructurado y documentación: diagramas de flujo estructurado, diagramas de HIPO o de Warnier/Orr. Todos usan la notación gráfica para describir sistemas o módulos. Los *diagramas de flujo estructurado*, también llamados diagramas de *Nassi-Schneiderman*, definen los módulos de un sistema usando las tres estructuras básicas de proceso, decisión e iteración. Cada una se ensambla de forma descendente para especificar la lógica de un módulo o sistema.

HIPO (entrada-proceso-salida jerárquica) también es un diagrama gráfico del sistema y está formado por una *tabla visual de contenido* que describe el sistema en general, y un conjunto de diagramas funcionales. Cada diagrama muestra la entrada, salida, pasos del proceso y flujo de los datos. Los diagramas de HIPO son muy adecuados para documentar un sistema.

Los *diagramas de Warnier/Orr* muestran en forma explícita las relaciones jerárquicas entre los procesos y los subprocesos. Para desarrollar un diagrama de Warnier/Orr, el analista trabaja hacia atrás, empezando con la salida del sistema y definiendo el sistema cada vez con más detalle. Los requerimientos de datos se añaden para cada proceso. Estos diagramas sencillos y fáciles de entender son una forma excelente de mostrar las relaciones entre los procesos que integran un sistema.

El aseguramiento de la calidad también incluye pruebas para garantizar que el sistema se desempeña de forma adecuada y que cumple con sus requerimientos. Los casos especiales de prueba son la *validación*, *verificación* y *certificación*. El propósito de la prueba es hallar errores, no el demostrar lo correcto de un sistema. Los *casos de prueba*, que usan datos reales o artificiales, son procesados por el software y se reportan los errores.

Se usan dos estrategias de prueba, ninguna de las cuales es ideal o suficiente. En la *prueba de código*, el analista desarrolla casos de prueba para ejecutar cada instrucción o ruta dentro de un programa. En la *prueba de especificación*, el analista examina las especificaciones del programa y entonces escribe los datos de prueba para determinar cómo opera el programa bajo condiciones específicas.

Los dos niveles de prueba son las pruebas parciales y la prueba del sistema. En la primera, el analista, usando alguna de las estrategias anteriores, examina los programas que conforman un sistema. Por el contrario, la prueba de sistemas pretende hallar cualquier discrepancia entre el sistema y sus objetivos originales.

Existen seis pruebas especiales que son: la prueba de carga máxima, almacenamiento, tiempo de ejecución, recuperación, proce-

dimiento y de factores humanos. Cada una se centra en hallar los defectos de operación en el sistema para prevenir su falla.

Tanto los datos reales como los artificiales se usan para probar el sistema. Algunas organizaciones guardan los datos en *bibliotecas de prueba* para garantizar que todos los sistemas relacionados pueden procesar un conjunto común de datos de prueba cuidadosamente preparados. Además de la prueba inicial, las bibliotecas de prueba proporcionan datos confiables para probar el sistema cuando evolucione.

Las fallas en la prueba se muestran rápidamente cuando el sistema se implanta. La implantación del sistema se estudia en el capítulo siguiente.

PREGUNTAS DE REPASO

1. ¿Qué es la confiabilidad de un sistema? Comente los enfoques de confiabilidad de un sistema, señalando las ventajas y desventajas de cada uno. ¿Qué enfoque prefiere? ¿Por qué?
2. ¿En qué difieren los errores de hardware y software?
3. ¿En qué categorías se dividen las actividades de mantenimiento de un sistema? ¿Cuál es la frecuencia relativa de cada una?
4. ¿Qué niveles de modularidad caracterizan al software? ¿Cómo varían los distintos niveles?
5. ¿Qué son los diagramas de estructura? Comente sus componentes. ¿Qué papel juegan los diagramas de estructura en el diseño de software?
6. ¿Cuáles principios guían el diseño de software? Para cada uno, describa brevemente la intención del principio. ¿Qué resultados pueden ocurrir si no se siguen los principios?
7. ¿Qué es acoplamiento y cohesión? ¿En qué difieren y en qué se parecen? Dé ejemplos de cada uno, usando una aplicación representativa de sistema de información.
8. Comente el concepto de diagramas de flujo estructurado. ¿Cuáles son los elementos o componentes de los diagramas de flujo estructurado?
9. ¿Qué es HIPO? Comente el concepto y sus componentes. ¿En qué difiere HIPO de los diagramas de flujo estructurado? ¿Y de los diagramas de Warnier/Orr?
10. Comente las características de los diagramas de Warnier/Orr para el desarrollo de sistemas. ¿Cuáles son las ventajas de los diagramas como herramientas de diseño comparadas con otros métodos en este capítulo?
11. ¿Qué es el aseguramiento de la calidad en lo que respecta al software? ¿Por qué es un aspecto importante de la ingeniería de sistemas?
12. Comente los tipos más comunes de aseguramiento de la calidad. Dé ejemplos de cada tipo.
13. ¿Cuál es la finalidad de la prueba de software? Explique con ejemplos.
14. ¿En qué difieren la prueba de código y la prueba de especificación? Explique el propósito de cada una.
15. Comente las seis pruebas especiales de un sistema. Explique el propósito de cada una. Dé ejemplos para mostrar el propósito de la prueba.
16. ¿Cuáles son los niveles de prueba de un sistema? Identifíquelos y explique cada uno.
17. ¿Deben usarse datos reales o artificiales durante las pruebas de un sistema? Explique su razonamiento.

PROBLEMAS DE APLICACIÓN

1. Elabore un diagrama de estructura que describa la situación siguiente, en la que los pedidos de clientes se procesan en una organización.

Todos los pedidos se identifican mediante un número único de pedido. Todos los artículos vendidos por la compañía también tienen un número único de producto.

El procedimiento del sistema que se describe será automatizado aplicando buenas prácticas de diseño modular. Un operador de pedidos recibirá los pedidos por teléfono mientras trabaja en una terminal en línea. Se tomará la información por teléfono, y se introducirá al sistema por medio de una terminal.

Cuando se recibe un pedido de un cliente en la compañía, se asigna el número consecutivo de pedido para identificar el pedido. Además, se introduce el número de cuenta del cliente en el pedido, junto con el nombre y la dirección. Si la persona que hace el pedido ya es un cliente regular de la empresa, su nombre, dirección y dirección de envío tienen que estar en el archivo, identificados por el número de cuenta. De ser así, la información debe poder recuperarse cuando el cliente proporcione el número de cuenta. Sin embargo, si el cliente es nuevo, el sistema pedirá al operador de la terminal que introduzca el nombre del cliente, su dirección, dirección de envío e información bancaria y de crédito.

Todos los artículos de un pedido se introducen mediante el número del producto. Si el cliente le dice al operador el número del producto, éste lo introduce a la terminal, junto con la cantidad a ordenar. A su vez, la descripción y costo del producto se muestran en la terminal junto con una anotación de que los artículos están ahora "pedidos por el cliente". Cuando el cliente no conoce el número del producto, el operador introduce la clase de producto, dependiendo de la descripción que proporcione el cliente. En la pantalla de la terminal aparecen todos los artículos de esa clase. El operador y el cliente eligen conjuntamente el artículo correcto y se introduce su número de producto en el sistema, como se describió en el párrafo anterior.

Al final del pedido, el sistema muestra un listado de todos los artículos y cantidades solicitadas, los costos individuales y totales por cada artículo y el costo total global del pedido.

Al final del día, se imprime una copia de cada pedido y se la manda al cliente. Además, se prepara un reporte que resume todos los pedidos del día. El reporte incluye el número de cliente, nombre, dirección y dirección de envío, al igual que el monto total de cada pedido. Al preparar el diagrama de estructura:

- a. Muestre todos los módulos del sistema, identificando cada uno con un nombre descriptivo de su función. Use la notación estándar.
 - b. Muestre la secuencia de llamadas entre módulos.
 - c. Muestre los datos transferidos entre módulos.
 - d. Anote cualquier hipótesis que haga al preparar el diagrama.
 - e. Use un buen diseño modular.
2. El proceso de actualización de archivos secuenciales es común en todos los sistemas de información automatizados, independientemente del medio de almacenamiento usado para guardar transacciones y el archivo maestro. Si el archivo de transacciones se clasifica igual que el archivo maestro, los pasos son bien conocidos:
 - Preparar el archivo maestro para su uso.
 - Preparar el archivo de transacciones para procesamiento.
 - Leer un registro del archivo maestro.
 - Leer un registro del archivo de transacciones.

Si las llaves del archivo maestro y el archivo de transacciones concuerdan, el registro maestro se modifica y se escribe de nuevo en la memoria y los nuevos registros maestro y de transacción se leen en la memoria.

Si la llave del archivo de transacciones es menor que la llave del archivo maestro, se crea un registro y se escribe en el archivo maestro y se lee el siguiente registro del archivo de transacciones.

Si la llave del archivo de transacciones es mayor que la llave del registro del archivo maestro, el registro maestro se escribe en el nuevo archivo, se lee el nuevo registro maestro y el proceso se repite.

El proceso anterior se repite hasta que no haya más transacciones o registros en el archivo maestro. Entonces se cierran los archivos.

- a. Desarrolle un diagrama de flujo estructurado para el procedimiento de proceso de datos descrito.
- b. Desarrolle un diagrama de Warnier/Orr para el procedimiento.
- c. ¿Cómo cambiarán los diagramas si se supone que
 - (1) ¿Hay más de una transacción para aplicarla a un registro específico del archivo maestro?
 - (2) ¿El archivo maestro se organiza con una organización de acceso directo?

3. Un patrón común de diseño que se usa cuando los individuos deben interactuar con un sistema en línea mediante pantallas, usa una serie de menús. Cada menú informa al usuario de las opciones que puedan ejecutarse en ese punto. En un sistema, el usuario entra introduciendo la palabra clave STARTUP (cualquier otra palabra clave es un error) y el sistema responde mostrando el menú principal del sistema. Las opciones del menú principal son la de añadir nuevos registros al archivo, editar los registros existentes, generar reportes, reindexar el archivo principal o salir del sistema.

Si el usuario elige la opción de reporte, se presenta otro menú a partir del cual el usuario puede elegir uno de cuatro reportes. Después de que el reporte ha sido seleccionado, se muestra otra pantalla preguntando al usuario si el reporte debe estar en orden alfabético, numérico o cronológico.

El usuario puede salir del sistema desde cualquier menú o puede regresar al siguiente menú de nivel superior. La salida se hace oprimiendo x o X. El movimiento de un menú a otro se hace oprimiendo m o M. Todas las opciones se eligen mediante un número (1, 2, 3, 4, etc.)

- a. Desarrolle una tabla visual de contenido de HIPO para este diseño.
 - b. Describa el proceso por medio de un diagrama de Warnier/Orr.
 - c. Muestre la entrada al sistema y el proceso de selección de menú en forma de diagrama de flujo estructurado.
4. Los documentos que producen las empresas para enviar mercancías a los clientes a menudo reciben el nombre de *documentos de envío*. Análogamente, el sistema que produce los documentos se conoce como un *sistema de documentos de envío*. Un sistema particular utiliza un archivo de envíos para producir los documentos apropiados para el transporte de mercancía a los clientes, los cuales se clasifican con un código único de dos caracteres que describe el tipo de cliente y forma de envío a usar. Si el código de envío es BK, DV o CN, el programa añade un nuevo registro de envío al archivo de transacciones de envíos, archivo que se crea para su uso posterior por otro módulo del sistema. Si el código de envío es DA y el destino es un código de planta de producción DA, o si el código de envío es HH y el destino es una planta de producción con código DA, también se añade un registro de transacción al archivo de transacciones de envíos. El otro caso donde el sistema debe crear un registro de transacción de envío es si el código de pedido es QZ5.

Para estas situaciones y los demás códigos, el sistema produce una factura (además de la creación de los registros de transacción de envío descritos). Las existencias en el archivo de inventario de mercancías se reduce en el número de unidades facturadas cuando se prepara el documento de envío. Si el número de artículo en el documento de envío no coincide con ningún número de artículo en el archivo de inventario, el archivo no se altera y se produce un mensaje de error. Sin embargo, el documento de envío se sigue preparando como ya se describió.

- a. Desarrolle un diagrama de HIPO mostrando la entrada, proceso y salida para el sistema de envío tal y como se ha descrito.
 - b. Muestre la lógica del diseño usando diagramas de Warnier/Orr.
 - c. Compare los diagramas desarrollados en *a* y *b*. ¿Qué ventajas ofrece un método sobre el otro? ¿Cuáles son las desventajas de cada método?
5. Un analista diseñó un sistema de informes a la dirección que proporciona un reporte de una sola página donde resume las ventas y gastos del día. Las actividades de cada uno de los tres departamentos principales se reporta por separado, así como el total de los otros departamentos juntos.

Además, el sistema reporta las ventas acumuladas durante ese año para cada departamento y en total. Lo hace recuperando las ventas del día anterior y añadiendo las ventas del día de hoy a ese total. Después se graba el nuevo total junto con la fecha de hoy.

El sistema se diseñó de tal forma que haya un registro grande de datos por cada día. En un año normal, el archivo contiene 365 días. Cada número de registro es el día numérico del año. Por ejemplo, el dato del primero de enero se guarda en el registro 1, el 31 de enero en el registro 31, el primero de febrero en el registro 32 y así sucesivamente. Cuando el usuario entra al sistema, hay que introducir la fecha actual. A su vez, el sistema traduce esto a un valor numérico.

Si la fecha de hoy es el 31 de enero, el sistema recuperará las ventas acumuladas hasta la fecha del registro 30 (para el 30 de enero) y se añaden las ventas del día 31 para formar un nuevo total acumulado hasta el 31 de enero. Ese total se guardará como parte del registro 31 pero el registro 30 no se modifica. Al final del año, el sistema se recicla. En otras palabras, después del 31 de diciembre, el sistema empieza a reusar el archivo comenzando con el registro 1 (para el primero de enero).

Desarrolle casos de prueba para determinar si el sistema recupera los datos del día anterior en forma adecuada. Especifique todas las combinaciones de datos necesarios para garantizar que la recuperación del día anterior trabaja adecuadamente. No hay que olvidar el año bisiesto, en el que hay 29 días en febrero y 366 días en el año.

6. Escriba el conjunto de los datos de prueba que crea necesarios para probar en forma adecuada el procedimiento de actualización de archivos descritos en la pregunta 4.

BIBLIOGRAFÍA

- BOEHM, B.W.: "Improving Software Productivity", *Computer*, 20, 9, septiembre de 1987, pp. 4357.
- BOEHM, B.W.: "Software Engineering"; *IEEE Transactions on Computers*, 25, 12, diciembre de 1976, pp. 1226-1241.
- BROOKS, F.P. JR.: *The Mythical Man Month*, Reading, MA: Addison Wesley, 1978.

- HECKEL, P.: *The Elements of Friendly Software Design*, NY: Warner Books, 1984.
- HIGGINS, D.A.: *Program Design and Construction*, Englewood Cliffs, NJ: Prentice-Hall, 1979.
- JACKSON, M.A.: *Principles of Program Design*, NY: Academic Press, 1975.
- LIENTZ, B.P. y E.B. SWANSON: *Software Maintenance Management*, Reading, MA: Addison Wesley, 1980.
- METZGER, P.W.: *Managing a Programming Project*, 2a. edición, Englewood Cliffs, NJ: Prentice-Hall, 1981.
- MYERS, G.J.: *The Art of Software Testing*, NY: Wiley-Interscience, 1979.
- MYERS, G.J.: *Software Reliability*, NY: John Wiley and Sons, 1976.
- NASSI, I. y B. SCHNEIDERMAN: "Flowchart Techniques for Structured Programming", *SIGPLAN Notices*, 8, 8, agosto de 1973, pp. 12-26.
- PARNAS, D.L., P.C. CLEMENTS y D.M. WEISS: "The Modular Structure of Complex Systems", *IEEE Transactions on Software Engineering*, 34, 3, marzo de 1985, pp. 259-266.
- PETERS, L.J.: *Software Design: Methods and Techniques*, NY: Yourdon Press, 1981.
- STEVENS, W.P., G.L. MEYERS y L.L. CONSTANTINE: "Structured Design", *IBM Systems Journal*, 13,2, mayo de 1974, pp. 115-139.
- WARNIER, J.D.: *Logical Construction of Programs*, NY: Van Nostrand Reinhold Company, 1974.
- WASSERMAN, A.I. y STEVEN GUTZ: "The Future of Programming", *Communications of the ACM*, 25, 3, marzo de 1982, pp. 196-206.
- YOURDON, E. y L.L. CONSTANTINE: *Structured Design*, Englewood Cliffs, NJ: Prentice-Hall, 1979.

15. Administración del proceso de implantación del sistema

GUÍA DE ESTUDIO

Usted entenderá bien cómo administrar el proceso de implantación de los sistemas de información cuando pueda responder a estas preguntas:

- ¿Qué aspectos deben considerarse en la fase de capacitación de los usuarios y los operadores?
- ¿Cómo varían los métodos de capacitación entre las organizaciones?
- ¿Qué aspectos de la conversión de sistemas son los más difíciles de administrar?
- ¿Existen estrategias de conversión variables?
- ¿Cómo aseguran los analistas de sistemas que la conversión se lleva a cabo de forma adecuada y precisa?
- ¿Porqué es necesaria la revisión después de la implantación cuando los sistemas se instalan?
- ¿Cuáles son los métodos que se usan para revisar los sistemas después de la implantación?



OBJETIVOS DEL CAPÍTULO

- Evaluar un programa de capacitación para determinar si es completo y si tiene posibilidad de éxito.
- Preparar un plan de conversión para obtener un nuevo sistema de información.
- Diseñar un procedimiento para la conversión de los datos y archivos ya existentes para utilizarlos en el nuevo sistema.
- Delinear un plan de revisión después de la implantación.

PALABRAS CLAVE

Bitácora de eventos
Conversión
Conversión directa
Enfoque piloto
Etapas

Evaluación del impacto
Gerente de conversión
Procedimientos de ejecución
Revisión después de la implantación
Sistemas paralelos

Vacaciones en casa

Linda Gianni ha estado pensando cómo tomará su personal la implantación del nuevo sistema en su oficina. Todos parecen estar entusiasmados con la computarización, aun cuando ninguno de los miembros del personal, excepto ella, han tenido experiencia directa con las computadoras. Después de todo, en la mayoría de los casos, ellos están familiarizados con estas máquinas, ya que tienen cónyuges o hijos que usan computadoras en el trabajo o en la escuela. Al existir computadoras virtualmente en toda empresa, banco, tienda y terminal aérea, los temores y las preocupaciones que hace algunos años la gente tenía con respecto a la computarización han desaparecido en gran medida. El personal reconoce que las computadoras son parte integral de las empresas, así que están a favor del cambio. Se dan cuenta que les ahorrarán tiempo y mejorarán el servicio.

El departamento de Linda es el responsable de mantener registros exactos de los 5000 vehículos en renta de la compañía; el nuevo sistema promete hacer más fácil el trabajo para ella y sus compañeros de trabajo. Cada registro individual mostrará la fecha y el precio de compra del vehículo, la historia del mantenimiento y los términos del convenio de renta. Además, el sistema eliminará la necesidad de procesar manualmente los estados de cuenta mensuales que se envían a los arrendatarios, procedimiento que el personal desea automatizar. Como gerente, Linda se preocupa por las necesidades y deseos de sus compañeros de trabajo, así como de las necesidades y deseos de la empresa. Por fortuna, el nuevo sistema tiene el potencial para satisfacer a todos.

Linda revisa de nuevo la carta del proveedor que muestra los planes de capacitación para su departamento.

El programa de capacitación para el nuevo sistema será de cuatro días:

- | | |
|--------------|---|
| Primer día: | conceptos de computación, incluyendo una introducción al hardware, un panorama de las funciones que realiza el software y procedimientos generales de procesamiento de datos. |
| Segundo día: | experiencia directa con los menús, funciones y características del sistema. |
| Tercer día: | creación de archivos, introducción de datos, preparación de reportes e impresión de salidas. |
| Cuarto día: | práctica supervisada de uso del sistema. |

Durante las sesiones de capacitación, el personal comenzará a construir archivos para su propio uso e introducirá los datos reales que ellos necesitan para procesar las cuentas. Esto asegurará que la capacitación sea realista, efectiva y también útil.

En vista de que se acerca el periodo de vacaciones, hemos tratado de preparar una programación que no interfiera con los planes vacacionales de los empleados.

Como resultado, hemos programado la capacitación para impartirse del 1 al 20 de diciembre, periodo de tres semanas que consideramos adecuado para las veinte personas que recibirán la capacitación.

Al final de la capacitación, daremos un receso hasta el 5 de enero, fecha en la cual adoptaremos el nuevo sistema. Todo el equipo se instalará y probará antes del primero de diciembre. Entre el 30 de diciembre y el 5 de enero probará el hardware, se harán ajustes al software o se corregirá cualquier dificultad que surgiera durante la capacitación. Ninguna de estas actividades interferirá con las operaciones regulares en su oficina.

Linda dejó la carta y empezó a pensar en las implicaciones de la programación propuesta. ¿Es una buena idea el receso de vacaciones?, se dijo a sí misma. Todos quieren el sistema en su lugar, pero tal vez la capacitación y la conversión debieran retrasarse hasta después de las vacaciones para que haya continuidad. Por supuesto, el proveedor probablemente no tenga ese espacio de tiempo disponible para la capacitación, dado un aviso tan precipitado.

También existe el problema de asegurarse que los registros se introduzcan de manera apropiada. Al tener que introducir los registros de 5000 vehículos por medio de las terminales, habrá muchas posibilidades de cometer errores u omisiones, recordó. ¿Será una buena decisión que el personal "practique" construyendo archivos con los datos reales? ¿No existe un gran peligro de equivocarse?

Al examinar la situación, Linda se preguntó: ¿cuál es el mejor método para dirigir la implantación? La programación propuesta, ¿dará por resultado una capacitación y conversión de la mejor calidad posible?

La implantación incluye todas aquellas actividades que tienen lugar para convertir del sistema anterior al nuevo. El nuevo sistema puede ser totalmente nuevo y reemplazar al sistema que hay, ya sea manual o automatizado; o bien puede ser una modificación importante de un

	CAPACITACIÓN DEL PERSONAL DE SISTEMAS	CAPACITACIÓN DEL PERSONAL USUARIO
Participantes	Personal de preparación de datos Personal de captura de datos Operadores de computadora	Todos los usuarios directos o indirectos
Temas principales	Uso de equipo Identificación de los problemas del equipo Procedimientos de ejecución de la computadora Programación de trabajos de la computadora Programación de actividades Mantenimiento del sistema	Uso del equipo (en caso necesario) Identificación de los problemas del equipo (en caso necesario) Familiarización con la aplicación Captura y codificación de los datos Manejo de datos: añadir registros eliminar registros editar registros Recuperación de información Uso de la información
TIPOS DE CAPACITACIÓN		
	Proporcionada por el vendedor:	Uso del equipo Operación directa Características y controles del software
	"En casa":	Igual que arriba, pero se puede adaptar a las necesidades del usuario. Se puede usar material audiovisual, manuales de capacitación especial o software especial.

FIGURA 15.1
Actividades de capacitación de sistemas.

sistema existente. En cualquier caso, la adecuada implantación es esencial para lograr un sistema confiable y que cumpla con las necesidades de la organización. Una implantación exitosa no garantiza el mejoramiento de la organización que use el nuevo sistema (eso es una cuestión de diseño), pero su instalación inadecuada lo impedirá.

Como lo muestra este capítulo, aun los mejores sistemas pueden debilitarse si los analistas que conducen la implantación no atienden cada detalle importante. Ésta es un área en la que los nuevos analistas de sistemas deben concentrar gran parte de su atención. Los *buenos analistas* pueden hacer una gran diferencia.

Este capítulo estudia los tres aspectos de la implantación: capacitación de personal, procedimientos de conversión y la revisión después de la implantación. En cada área, se estudian los elementos particula-

res de ese aspecto, así como los métodos de manejo de cada situación de forma eficiente y efectiva.

CAPACITACIÓN

Aun los sistemas técnicamente elegantes y bien diseñados pueden tener éxito o fracasar debido a la forma en que se operan y usan. Por lo tanto, la calidad de la capacitación recibida por el personal relacionado con el sistema ayuda u obstruye, y puede llegar a impedir, la implantación exitosa de un sistema de información. Aquellos que estén asociados con el sistema o afectados por el mismo deben conocer con detalle cuáles serán sus papeles, cómo pueden usar el sistema y qué hará o no hará el sistema. Tanto los operadores como los usuarios del sistema necesitan capacitación (Fig. 15.1).

Capacitación de operadores de sistemas

Muchos sistemas dependen del personal del centro de cómputo, el cual es responsable de mantener al equipo funcionando, así como de proporcionar el servicio de apoyo necesario. Su capacitación debe asegurar que puedan manejar todas las operaciones posibles, tanto rutinarias como extraordinarias. La capacitación del operador también debe contemplar al personal de captura de datos.

Si el sistema necesita la instalación de nuevo equipo, por ejemplo, un nuevo sistema de computación, terminales especiales, o equipo distinto para la captura de datos, la capacitación de los operadores debe incluir aspectos tan básicos como saber prender el equipo, usarlo, apagarlo, y también un conocimiento de lo que es su operación y uso normales. También debe capacitarse a los operadores acerca de los desperfectos más comunes, cómo reconocerlos y qué pasos llevar a cabo cuando ocurran. Como parte de su capacitación, se debe dar a los operadores una lista de formas de resolver los problemas y que identifique los posibles problemas y solución, así como los nombres y números telefónicos de las personas a quién buscar cuando surjan problemas inesperados.

La capacitación también necesita la familiarización con los *procedimientos de ejecución*, lo cual implica trabajar a través de la sucesión de actividades necesarias para usar un nuevo sistema. Estos procedimientos permiten a los operadores de la computadora familiarizarse con las acciones que deben realizar (como montar discos o cintas magnéticas, copiar archivos, cambiar las formas de impresión o hacer funcionar sistemas de información) y saber cuándo deben ocurrir dichas acciones. Además, sabrán cuánto tiempo le llevará a las aplicaciones correr bajo condiciones normales. Esta información es importante, tanto para que los usuarios puedan planear su trabajo como para identificar los sistemas que corran más rápido o más lento de lo esperado —una señal que usualmente indica problemas con la corrida—.

Capacitación de usuarios

La capacitación de usuarios puede implicar el uso de equipo, particularmente en el caso en que, digamos, una microcomputadora esté en uso y la persona en cuestión sea al mismo tiempo operador y usuario. En estos casos, primero se debe capacitar a los usuarios sobre cómo operar el equipo. Preguntas que pueden parecer triviales para el analista, por ejemplo, cómo prender una terminal, cómo introducir un diskette en una microcomputadora, o cuándo se debe apagar el equipo sin el peligro de perder datos, son problemas importantes para los nuevos usuarios que no están familiarizados con las computadoras.

La capacitación de los usuarios también debe incluir la identificación de los problemas, determinando si el problema que surge es causado por el equipo o el software o por algo hecho por ellos al usar el sistema. Al incluir una guía de identificación de problemas en la documentación de los sistemas, se obtendrá una referencia útil por mucho tiempo después de terminar el periodo de capacitación. No hay nada más frustrante que trabajar con un sistema, hallar un problema, y no ser capaz de determinar si es error del usuario o si es algo que tiene que ver con el propio sistema. El lugar para prevenir la frustración es durante la capacitación.

La mayor parte de la capacitación del usuario tiene que ver con la operación del sistema en sí. La capacitación en la codificación de datos enfatiza los métodos a seguir en la captura de datos a partir de las transacciones, o en la preparación de datos necesarios para las actividades de apoyo a las decisiones. Por ejemplo, en un sistema de contabilidad, puede ser importante traducir los nombres de los clientes en números de cuenta de clientes que se introducen como parte de la transacción contable (Fig. 15.2). Los usuarios deben saber cómo determinar el número de cuenta del cliente, que tiene cuatro dígitos y que no contiene caracteres alfabéticos.

Las actividades de manejo de datos que reciben la mayor atención en la capacitación de usuarios son la captura de datos (cómo guardar nuevas transacciones), la edición de datos (cómo modificar datos grabados previamente), la formulación de consultas (cómo localizar registros específicos u obtener respuestas a preguntas) y el borrado de registros de datos. El grueso del uso del sistema implica este conjunto de actividades, lo cual quiere decir que la mayor parte del tiempo de la capacitación se dedicará a esta área.

De vez en cuando, los usuarios deberán preparar discos, meter papel en las impresoras, cambiar las cintas de las impresoras. Ningún programa de capacitación está completo sin dedicarle tiempo a las actividades de mantenimiento de los sistemas. Si una microcomputadora o sistema de captura de datos usa discos, los usuarios deben estar preparados para formatear y probar discos. También deben realizar cambios de cintas de impresoras, limpieza del equipo y otros mantenimientos rutinarios. No es suficiente el simplemente incluir esta

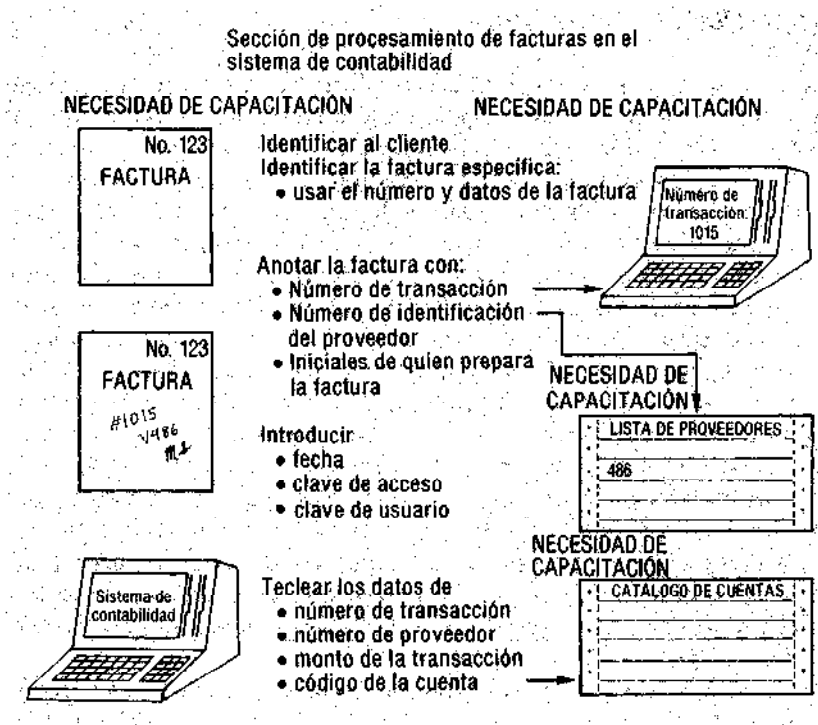


FIGURA 15.2
Necesidades de capacitación para la captura de datos de un sistema de contabilidad.

información en un manual, aunque esto es esencial para referencias futuras.

Como lo muestra el párrafo anterior, hay dos aspectos de la capacitación de los usuarios: la familiarización con el sistema de procesamiento en sí (es decir, el equipo usado para la captura y procesamiento de datos) y la capacitación en el uso de la aplicación (es decir, el software que acepta los datos, los procesa y produce los resultados). La debilidad de cualquier aspecto de la capacitación trae la posibilidad de llegar a situaciones embarazosas que producirían en el usuario frustración, errores, o ambos. Una buena documentación, aunque esencial, no reemplaza la capacitación. No hay sustituto para la operación directa del sistema mientras que se aprende su uso.

Métodos de capacitación

La capacitación de operadores y usuarios se puede obtener de distintas formas. Las actividades de capacitación pueden llevarse a cabo en las instalaciones del proveedor, en locales rentados, por ejemplo, en hoteles o campus de universidades, "en casa", o en las instalaciones de la empresa. Los métodos y contenido de la capacitación varían a menudo, dependiendo de la fuente y del lugar de la capacitación.

Capacitación por el proveedor

A menudo, la mejor fuente de capacitación para un equipo es el proveedor de éste. La mayoría de los proveedores ofrecen programas educacionales amplios como parte de sus servicios. En algunos casos hay un cargo, pero muchas veces la capacitación es gratis. Por ejemplo, IBM ofrece cursos complementarios de dos o tres días a los compradores de muchas de sus minicomputadoras y equipos grandes. Los cursos, impartidos por instructores con experiencia y personal de ventas, cubren todos los aspectos del uso del equipo, desde cómo prenderlo y apagarlo hasta cómo guardar y eliminar información y también cómo atender las fallas. Esta capacitación es directa, por lo que los participantes realmente usan el equipo en presencia de los instructores. Si surgen preguntas, éstas se pueden contestar en seguida. Puesto que el objetivo del sistema es la capacitación, generalmente no hay prisa por dejar la capacitación de lado para que comience el uso productivo del sistema. La capacitación llevada a cabo en la empresa puede precipitar un riesgo del que debe protegerse el personal de instalación.

Si se va a instalar un software especial, tal como un paquete de teleproceso o un sistema de administración de bases de datos, es preferible mandar personal a cursos cortos fuera de la empresa para obtener una capacitación amplia, en vez de una capacitación para el servicio. Estos cursos, que generalmente se imparten cobrando honorarios, se dan al personal de muchas empresas que compran o usan el mismo sistema. Los beneficios de compartir preguntas, problemas y experiencias con personas de otras compañías son muy importantes. Los contactos personales hechos durante las sesiones frecuentemente duran años, además se comparten continuamente la información y por tanto se benefician ambas partes. Los cursos cortos a menudo implican tiempo y costos adicionales para viajar a otras ciudades.

Capacitación "en casa"

La ventaja de ofrecer capacitación para el sistema en el lugar de la empresa es que la instrucción se puede unir a la organización donde se ofrece y enfocarse a procedimientos especiales usados en ese marco, los planes de la organización para su crecimiento y cualquiera de los problemas que hayan surgido. A menudo, los proveedores o las compañías de capacitación ofrecen tarifas más económicas y que permiten a la organización involucrar a más personal en el programa de capacitación que el que es posible cuando se necesita viajar.

También hay desventajas. El simple hecho de que los empleados estén en su propio medio es una distracción, ya que las llamadas telefónicas y las emergencias pueden interrumpir las sesiones de capacitación. Además, cuando los proveedores externos van a una empresa, pueden presentar cursos que enfatizan los conceptos generales, pero que carezcan de una práctica directa que sea suficiente. El coordinador de la capacitación debe reconocer esta posibilidad y tratarla de

antemano para garantizar que el contenido del curso satisfaga las necesidades de operación.

La capacitación "en casa" también se puede ofrecer por medio de material instruccional adquirido de manera especial. Es posible rentar o adquirir una variedad de programas profesionales de enseñanza de varios temas por medio de empresas especializadas, tales como Edu-tronics (McGraw-Hill, Inc.); Deltak, Inc.; Professional Development, Inc., y Learning Corporation of America. Otros proveedores ofrecen materiales impresos o audiovisuales de instrucción programada que son autoinstructivos o bien complementan otras actividades de capacitación.

Sin embargo, no hay sustituto para la experiencia directa. Los manuales de capacitación son aceptables para la familiarización, pero las experiencias de usar realmente el equipo, hacer y corregir errores y hallar situaciones inesperadas son la mejor y más duradera forma de aprender.

Los manuales de capacitación generalmente toman uno de dos enfoques. Algunos hacen que el usuario trabaje distintas actividades paso por paso. Por ejemplo, la lista de la figura 15.3 presenta los pasos necesarios para implementar un sistema, usando discos flexibles. Cada paso se enlista en el orden adecuado.

El otro enfoque común consiste en crear un caso de estudio que incluya todas las situaciones que se hallan frecuentemente, las cuales puedan ser manejadas por el sistema y que el usuario debería conocer. Los usuarios tienen que usar el sistema para llegar a las situaciones reales; es decir, introducir los datos en la forma pedida, procesarlos y preparar los informes. Si el sistema está orientado hacia las consultas, el caso de estudio pedirá a los usuarios que preparen y reciban respuestas a las consultas. La figura 15.4 incluye un caso de estudio parcial que capacitará a las personas para usar un sistema de contabilidad. Se incluyen ejemplos de datos y transacciones individuales, así que las personas reciben una retroalimentación inmediata de lo correcto de sus acciones. Si los resultados que obtienen no se equiparan con los de la guía de estudios, los usuarios sabrán que cometieron errores.

Durante la capacitación, el personal de sistemas debe mantenerse atento a los comentarios hechos por los usuarios y a los problemas que éstos puedan hallar. A pesar de que se hayan realizado con anterioridad estudios de los factores humanos, con la intención de detectar dificultades, algunos problemas pueden ocurrir sólo hasta que los usuarios no experimentados interactúan directamente con el sistema. Aun con las pruebas, es posible que surjan durante la capacitación manejos incorrectos del teclado al introducir los datos, transacciones inesperadas o formas no usuales de preparar transacciones. El instructor debe asegurarse de involucrar al personal de sistemas cuando se hallen errores en el diseño, a la vez que tiene que ayudar a los usuarios que son renuentes a cambiar sus viejas formas por los nuevos métodos

LISTA DE VERIFICACIÓN PARA USAR EL SISTEMA POR PRIMERA VEZ

- ____ 1. Encienda la computadora.
- ____ 2. Inserte el diskette maestro en la unidad 1.
- ____ 3. Usted deberá recibir el símbolo de comando "A>", el cual indica que el sistema está listo para la introducción de datos.
- ____ 4. Escriba el comando "DIR" para listar los archivos en el disco. Verifique que aparezcan los nombres de 14 archivos.
- ____ 5. Inserte un diskette en blanco en el drive 2.
- ____ 6. Escriba el comando "FORMAT" que prepara el disco en blanco para su uso. Cuando el sistema pida la unidad a formatear, escriba "2". El proceso de preparación del diskette se llevará aproximadamente 1 minuto.
- ____ 7. Pruebe el disco recién formateado, usando el comando "DISKTEST".
- ____ 8. Copie el contenido del disco maestro al nuevo disco, usando el comando "COPY DISK 1 TO DISK 2".
- ____ 9. Copie el sistema operativo al nuevo disco, usando el comando "COPY SYSTEM TO DISK 2".
- ____ 10. Ahora está usted listo para comenzar a usar el nuevo sistema. Retire el disco maestro. Use ahora la copia que acaba de hacer.

FIGURA 15.3

Lista de verificación de la capacitación para preparar un sistema pequeño para su uso inicial.

necesarios para usar el sistema. Por supuesto, en primer lugar el instructor debe estar seguro de que los nuevos métodos son necesarios y que realmente representan una mejoría con respecto a los métodos en uso.

Comentario al margen

Perspectivas de la gerencia con respecto a la capacitación

Usted podría pensar que los gerentes corporativos tratarían de controlar los gastos asociados con los nuevos sistemas de información reduciendo la capacitación. Sin embargo, los buenos gerentes ven la capacitación como una valiosa inversión.

En muchos esfuerzos de implantación, los gerentes corporativos muestran gran interés en la capacitación de su personal. Aunque no estén presentes durante la preparación del lugar o cuando se instala el nuevo equipo de cómputo, al desarrollarse la capacitación están presentes con frecuencia, observando y escuchando. Su presencia es en sí una garantía de la importancia de la capacitación. También quieren ver si a "su gente" le va bien durante la capacitación.

¿Cuál es la razón de tal interés? Los gerentes eficientes saben que ningún sistema será eficiente a menos de que su personal lo use sabiamente y adecuadamente, para lo cual se necesita una buena capacitación. Además, de esta forma, el personal aprenderá acerca de prácticas efectivas que el equipo de instructores del proveedor ha observado en diversas compañías, así como prácticas exitosas que la compañía quisiera adoptar. Con la capacitación, los gerentes y los empleados pue-



APLICACIÓN DE MUESTRA PARA SU NUEVO SISTEMA DE CONTABILIDAD

Para familiarizarlo con el sistema que ha adquirido, desarrollaremos un ejemplo de aplicación en el que usted podrá introducir datos, recuperar información mediante respuestas a consultas y producir reportes. Las actividades que desarrollará en el ejemplo serán análogas a las que usted hará cada día de uso normal del sistema.

Siga cada uno de los pasos descritos abajo. Usted no puede dañar el equipo aun cuando cometa un error, así que no tenga miedo de intentar algo aunque dude si es correcto o no.

1. Comience el trabajo diario escribiendo el comando:

STARTUP (RETURN)

(La tecla (RETURN) puede estar señalada como (ENTER) en su terminal).

Con esto, el sistema comienza y se prepara para aceptar sus registros

2. Lo primero que le pregunta es la fecha actual. Para esta sesión, introduciremos el dato de 15 de abril de 1991. Introduzca la fecha mediante el formato:

04-15-1991

Use la tecla de retroceso (la cual usualmente se halla en la esquina superior derecha del teclado) para regresar y volver a teclear un número si desea cambiar lo introducido.

A cada transacción introducida durante el día se le anotará automáticamente esta fecha para referencias posteriores.

3. El menú principal aparece como se muestra en la página anterior. Observe que en la parte inferior de la pantalla hay un mensaje intermitente:

EL DISKETTE NO HA SIDO PREPARADO PARA SU COMPAÑÍA

El mensaje le dice que la información de su compañía, tal como el nombre y la dirección, no han sido introducidos y que el sistema no se puede usar hasta que se proporcione dicha información. Usted debe dar esa información solamente al principio, cuando el sistema se use por primera vez. Esta información será grabada y se utilizará por el sistema cada vez que la necesite.

A continuación inicializaremos el sistema.

El número 5 del menú llamará a la rutina para inicializar la información de la compañía. Oprima el número 5 y después la tecla RETURN. Teclee la información solicitada en los espacios en blanco que aparecen en la pantalla:

BOULDER MANUFACTURING CO.

131

JAMES STREET

ATLANTA

GEORGIA

30303

Verifique sus datos con la información anterior para verificar la ortografía y la escritura con mayúsculas. Esta información se imprimirá en todos los reportes, informes y cheques que produzca el sistema. Use las teclas de flechas para mover el cursor (el punto parpadeante de luz que señala dónde aparecerá cada carácter de los datos) hacia arriba y hacia abajo en la pantalla. Teclee sus correcciones exactamente encima de los datos existentes y los cambios aparecerán en la pantalla.

FIGURA 15.4

Narración del enfoque por casos de la capacitación en sistemas.

- Oprima la tecla (ESC) (escape) cuando haya terminado.
4. A continuación aparece una forma que le pide a usted los datos del año fiscal y el fin del periodo para la compañía. Su periodo contable puede tener cualquier longitud. Para nuestra compañía ejemplo, introduzca la siguiente información después de los encabezados de los datos que se encuentran en la pantalla:

FECHA ACTUAL : 04-15-1991
FIN DEL PERIODO : 04-30-1991
ÚLTIMO FIN DE PERIODO : 03-31-1991
FIN DEL AÑO FISCAL : 12-31-1991

De nuevo, use las teclas de flecha para mover el cursor cuando desee hacer correcciones. Oprima (ESC) cuando haya terminado y aparecerá de nuevo el menú principal.

 5. El siguiente paso...

FIGURA 15.4
Continuación.

den ir más allá del dominio del nuevo sistema y aprender más sobre su negocio.

Los gerentes maduros tienen buena razón en ver la capacitación como una inversión, no como un gasto.

CONVERSIÓN

La conversión es el proceso de cambiar el sistema anterior al nuevo. En esta sección estudiaremos los métodos para desarrollar la conversión de sistemas y los procedimientos empleados para asegurarse que se lleva a cabo adecuadamente.

Métodos de conversión

Existen cuatro métodos para llevar a cabo una conversión de sistemas (Tabla 15.1). Cada método debe ser considerado a la luz de las ventajas que ofrece y los problemas que puede ocasionar. Sin embargo, muchas situaciones dictan el uso de un método sobre los demás, aun cuando los otros puedan ser más benéficos. En general, la conversión de sistemas debe realizarse lo más rápidamente posible. Los periodos de conversión largos aumentan la posible frustración y la dificultad de la tarea para las personas implicadas, incluyendo tanto a los analistas como a los usuarios.

Sistemas paralelos

El método más seguro para convertir un sistema anterior a uno nuevo es el de utilizar ambos sistemas en *paralelo*. Con este enfoque, los usuarios siguen operando el sistema anterior de la forma acostumbrada, pero también comienzan a usar el sistema nuevo. Este método es el enfoque de conversión más seguro, ya que garantiza que, en caso de surgir problemas, como errores en el procesamiento o incapacidad

TABLA 15.1 Métodos para la conversión de sistemas

Método	DESCRIPCIÓN	VENTAJAS	DESVENTAJAS
Sistemas paralelos	El sistema anterior se opera junto con el nuevo.	Ofrece la máxima seguridad. Se puede recurrir al sistema anterior si se hallan errores en el nuevo o si ocurren problemas de uso.	Duplica los costos de operación. El nuevo sistema puede no ser juzgado justamente.
Conversión directa.	El sistema anterior se reemplaza por el nuevo. La organización confía completamente en el nuevo sistema.	Obliga a los usuarios a que hagan trabajar el nuevo sistema. Hay beneficios inmediatos de los nuevos métodos y controles.	No hay otro sistema al cual recurrir si surgen dificultades con el nuevo. Requiere de la más cuidadosa planeación.
Enfoque piloto.	Se implanta una versión de trabajo del sistema en una parte de la organización. Con base en la retroalimentación, se hacen cambios y el sistema se instala en el resto de la organización mediante uno de los demás métodos.	Proporciona experiencia y prueba directa antes de la implantación.	Puede dar la impresión de que el nuevo sistema no es confiable ni está libre de errores.
Por etapas.	Se implanta el sistema de manera gradual a todos los usuarios.	Permite a los primeros usuarios aprovechar las ventajas del sistema. Permite la capacitación y la instalación sin uso innecesario de recursos.	Un largo periodo de instalación provoca la duda en el usuario de si el proyecto marcha bien (demasiado entusiasmo) o mal (resistencia y falta de un juicio justo).

de manejar ciertos tipos de transacciones en el nuevo sistema, la organización pueda regresar al sistema anterior sin pérdida de tiempo, ingresos o servicio.

Las desventajas del enfoque de los sistemas paralelos son significativas. En primer lugar, los costos del sistema se duplican, ya que existen dos sistemas. En algunos casos es necesario contratar personal temporal para operar ambos sistemas en paralelo. En segundo lugar, el hecho de que los usuarios sepan que es posible regresar a las formas antiguas puede ser una desventaja si existe una resistencia potencial al cambio o si los usuarios prefieren el sistema anterior. En otras palabras, el sistema nuevo puede no tener un juicio justo.

Después de todo, el método de conversión de los sistemas paralelos ofrece el plan de implantación más seguro si las cosas van mal, pero los costos y riesgos de un juicio justo no pueden ser pasados por alto.

Conversión directa

El método de *conversión directa* transforma el sistema anterior al nuevo de manera abrupta, a veces en un fin de semana o incluso durante una noche. Se usa el sistema anterior hasta un día de conversión ya planeado, en el cual es reemplazado por el sistema nuevo. Éste fue el método de conversión sugerido por el proveedor en el ejemplo del comienzo del capítulo. No hay actividades paralelas. Si el analista debe hacer el cambio y desea asegurarse de que el sistema nuevo reemplaza completamente al anterior, de manera que los usuarios no cuenten con los métodos anteriores, la conversión directa cumplirá con este fin. Psicológicamente, obliga a todos los usuarios a hacer trabajar el sistema nuevo, pues ellos no tienen otro sistema al cual recurrir.

La ventaja de no contar con un sistema de respaldo puede convertirse en una desventaja si surgen problemas serios con el sistema nuevo. En algunos casos, las organizaciones llegan a parar las operaciones cuando se presentan los problemas para que las dificultades puedan ser corregidas.

Una organización asignó a todo su personal de contabilidad para capturar datos y comenzar un nuevo sistema automatizado. La tarea llevó aproximadamente tres semanas, tiempo durante el cual ninguna de las operaciones contables regulares que debían convertirse al nuevo sistema se realizaron. En consecuencia, el trabajo sufrió un retraso de tres semanas. Sin embargo, tal retraso era esperado, así que la gerencia había planeado autorizar tiempo extra y la contratación de personal temporal para recuperarse después de la conversión. Cerca de dos días antes de llevarse a cabo la conversión directa, un gerente se dio cuenta de que el departamento de contabilidad no había planeado conservar los datos para establecer una historia de los estados de cuenta, así que detuvo la conversión. Como resultado, el equipo de contabilidad tuvo que recuperar el trabajo de tres semanas y volver a programar la conversión para un mes después, cuando muchos de los pasos previos tuvieron que repetirse. El sistema se implantó finalmente tres meses más tarde, después de mucho trabajo adicional, tiempo extra y la frustración del equipo debido a la forma en que se condujo la conversión.

Parar la conversión fue una medida particularmente drástica. Habría sido peor si los pasos se hubieran dado debido a problemas técnicos que necesitaran corrección. Si los usuarios saben que un sistema se detiene una vez por dificultades, ellos no tendrán toda la confianza de que el sistema será confiable, aun cuando los analistas les digan que se han corregido los problemas. El tiempo que lleva rehacer

el trabajo que fue interrumpido por la conversión puede ser largo y costoso y es posible que cuente como tiempo perdido que nunca se recupere.

La conversión directa necesita de una planificación cuidadosa. Las sesiones de capacitación deben ser programadas y mantenidas. La instalación del equipo tiene que ser a tiempo, con un rango amplio de días para corregir cualquier dificultad que pueda surgir. La preparación del lugar debe terminarse antes de hacer la conversión.

Las conversiones directas son algo común, sobre todo con sistemas comprados o de operación inmediata. Por ejemplo, la administración de un hotel decidió instalar un sistema automatizado de reservaciones. Todo el sistema se implantó durante una semana, en la cual el sistema de computadora se preparó, se cargó el software y se probó el sistema. Durante esos siete días, un equipo de instructores trabajó con todo el personal de contabilidad y de recepción para familiarizarlos con la operación y uso del sistema. Estas actividades se llevaron a cabo de lunes a sábado. El domingo, se reunió a todo el personal para capturar las reservaciones, cargos de clientes e información contable en el sistema nuevo, de manera que coincidiera con el sistema anterior. La tarde del domingo, después del cierre del día, se comenzó a usar el sistema nuevo y se utilizó de forma permanente. Se retiraron los archivos de reservación en papel anteriores y las cajas registradoras, y las máquinas de contabilidad se reemplazaron por las terminales. El sistema nuevo cobró vida a la medianoche del domingo. No había un sistema antiguo al cual recurrir.



Comentario al margen

La preparación contribuye a realizar el cambio

La conversión directa es una forma común de introducir nuevas aplicaciones. Si se dirige adecuadamente, los riesgos que implica el no tener el sistema anterior al cual recurrir serán mínimos. Es necesaria una aplicación bien diseñada que trabaje adecuadamente para poder estar seguros, pero eso no es suficiente.

La conversión directa funciona mejor cuando el equipo de conversión anticipa la aparición de problemas y está listo para enfrentarse a ellos. Un equipo de apoyo, listo para entrar en acción al menor aviso, es esencial. Tal vez sea posible que se disponga de un asistente para ofrecer apoyo a un empleado de menudeo que use un sistema de punto de venta por primera vez. O bien, que un instructor pueda ayudar al empleado a registrar los huéspedes de un hotel durante las primeras tardes después de la conversión. La gerencia debe participar durante la conversión para que, si surgen preguntas sobre el procedimiento, se pueda dar una respuesta inmediata.

Conviene recordar que el experimentar problemas y preguntas durante la conversión no origina una crisis, ¡pero la incapacidad de